TODOS

TODO 1: Check if section can be removed 9
TODO 2: Token vs token id 36
TODO 3: Example copilot 40
TODO 4: Mention in evaluation that all LLMs cannot complete the pydantic example previously, cause biased 40
TODO 5: List all design components and classes in this section, to provide an overview 44
TODO 6: Reference codellama tokenizer -> llama2 tokenizer -> llama1 tokenizer -> sentencepiece implementation: https://arxiv.org/pdf/2307.09288°, https://arxiv.org/pdf/1808.06226° 47
TODO 7: Streamline input_ids and other variable names, etc 50
TODO 8: Talk about scope, so when can items be removed? in design 54
TODO 9: Define process from retrieval, context, comment, prompt clearly 57
TODO 10: Move to evaluation? 66
TODO 11: Simpler languages like LISP easier to integrate auto completion, as less syntactical sugar? (example where instead of object.method it uses {} in python) 69

Heidelberg University
Institute of Computer Science
Artificial Intelligence for Programming (AIP)

Master's Thesis

Interrupt-Guided Neural Code Completion with Static Analysis of Dependencies

Name:	Niklas Loeser
Matrikelnummer:	3737339
Betreuer:	Prof. Dr. Artur Andrzejak°
Datum der Abgabe	2024-09-30

I hereby certify that I have written the work myself and that I have not used any sources or aids other than those specified and that I have marked what has been taken over from other people's works, either verbatim or in terms of content, as foreign. I also certify that the electronic version of my thesis transmitted completely corresponds in content and wording to the printed version. I agree that this electronic version is being checked for plagiarism at the university using plagiarism software.

Heidelberg, the 2024-08-06

_____

Niklas Loeser

Abstract

- English -

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aeque doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguique possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et aut officiis debitis aut rerum necessitatibus saepe eveniet, ut et voluptates repudiandae sint et molestiae non recusandae. Itaque earum rerum defuturum, quas natura non depravata desiderat. Et quem ad me accedis, saluto: 'chaere,' inquam, 'Tite!' lictores, turma omnis chorusque: 'chaere, Tite!' hinc hostis mi Albucius, hinc inimicus. Sed iure Mucius. Ego autem mirari satis non queo unde hoc sit tam insolens domesticarum rerum fastidium. Non est omnino hic docendi locus; sed ita prorsus existimo, neque eum Torquatum, qui hoc primus cognomen invenerit, aut torquem illum hosti detraxisse, ut aliquam ex eo est consecutus? – Laudem et caritatem, quae sunt vitae sine metu degendae praesidia firmissima. – Filium morte multavit. – Si sine causa, nollem me ab eo delectari, quod ista Platonis, Aristoteli, Theophrasti orationis ornamenta neglexerit. Nam illud quidem physici, credere aliquid esse minimum, quod profecto numquam putavisset, si a Polyaeno, familiari suo, geometrica discere maluisset quam illum etiam ipsum dedocere. Sol Democrito magnus videtur, quippe homini erudito in geometriaque perfecto, huic pedalis fortasse; tantum enim esse omnino in nostris poetis aut inertissimae segnitiae est aut fastidii delicatissimi. Mihi quidem videtur, inermis ac nudus est. Tollit definitiones, nihil de dividendo ac partiendo docet, non quo ignorare vos arbitrer, sed ut.

Abstract

- German -

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aeque doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguique possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et aut officiis debitis aut rerum necessitatibus saepe eveniet, ut et voluptates repudiandae sint et molestiae non recusandae. Itaque earum rerum defuturum, quas natura non depravata desiderat. Et quem ad me accedis, saluto: 'chaere,' inquam, 'Tite!' lictores, turma omnis chorusque: 'chaere, Tite!' hinc hostis mi Albucius, hinc inimicus. Sed iure Mucius. Ego autem mirari satis non queo unde hoc sit tam insolens domesticarum rerum fastidium. Non est omnino hic docendi locus; sed ita prorsus existimo, neque eum Torquatum, qui hoc primus cognomen invenerit, aut torquem illum hosti detraxisse, ut aliquam ex eo est consecutus? – Laudem et caritatem, quae sunt vitae sine metu degendae praesidia firmissima. – Filium morte multavit. – Si sine causa, nollem me ab eo delectari, quod ista Platonis, Aristoteli, Theophrasti orationis ornamenta neglexerit. Nam illud quidem physici, credere aliquid esse minimum, quod profecto numquam putavisset, si a Polyaeno, familiari suo, geometrica discere maluisset quam illum etiam ipsum dedocere. Sol Democrito magnus videtur, quippe homini erudito in geometriaque perfecto, huic pedalis fortasse; tantum enim esse omnino in nostris poetis aut inertissimae segnitiae est aut fastidii delicatissimi. Mihi quidem videtur, inermis ac nudus est. Tollit definitiones, nihil de dividendo ac partiendo docet, non quo ignorare vos arbitrer, sed ut.

Contents

1 Introduction 1
1.1 Motivation 1
1.2 Goals 2
1.3 Contributions 2

2 Background & Related Work 4
2.1 Causal Language Models 4
2.2 Text Generation Strategies 6
2.3 Controllable Generation 7
2.4 Torch 10
2.5 Transformers Library 11
2.6 Language Server Protocol 16
2.7 Related Work 24

3 Design 26
3.1 Requirements 26
3.2 Analysis 28
3.3 Interrupt-based meta-strategy 29
3.4 Language Server Protocol aided generation 33
3.5 Dataset 36

4 Implementation 43
4.1 Language Server Protocol aided generation 43
4.2 Dataset Implementation 60

5 Results 69
5.1 Collection method 69
5.2 Possible configurations 69
5.3 Results 69

6 Evaluation 70
6.1 Research Questions 70
6.2 Evaluation difficulties 70
6.3 Evaluation 70

7 Conclusion 71
7.1 Recap 71
7.2 Outlook 71

Acronyms 72

Bibliography 73

Listings 77
List of Figures 77
List of Tables 77
List of Code 77
List of Examples 79

Appendix 80
Language Server Protocol 80
DependencyEval Dataset 81

1 Introduction

Code completion provides code suggestions to developers extending their incomplete code. The suggestions used multimodal data sources to combine a variety of information.

It is one of the most used productivity tools in Integrated Development Environments (IDEs). Every major IDE from VSCode, Jetbrains IDEs, Eclipse, Neovim and Emacs with plugins support code completion. It reduces the mental burden of remembering method names, properties, their types and associated information, such as documentation and if the item is deprecated. In addition it saves time by reducing written boilerplate code by suggesting code snippets. Using the code completion saves time spent searching for corresponding documentation outside of the IDEs.

As Amann et al.[1] found, code completion is the most frequently used tool inside IDEs with a frequency of 19.7%. Moreover, developers spend 22.4% of their time inside IDEs navigating code, in contrast to 28.5% editing and executing the code. They hypothesize that a significant amount of time navigating code is spent due to a lack of understanding of the code. Moreover, 39.8% of the total time is spent outside the IDE, which likely includes time spent searching for documentation.

Good code completion is a crucial tool for developers, reducing time spent in total and reducing the time spent outside the IDE, further increasing the focus of the developer.

1.1 Motivation

Neural code completion tools, such as GitHub Copilot, have seen a massive adoption in the software development community.

Such tools promise to complement typical code completion tools by providing context-aware code completions spanning multiple lines. These multi-line completions reduce the need to write boilerplate code further, thus improving the productivity.

While typical code completion tools use static analysis to provide completions, neural code completions tools use causal language models trained on code to predict the next tokens. As Ziegler et al. [2] have shown, the completions provided by Copilot serve as a template to tinker with.

Even if the completions provide a starting point, this work argues that current neural code completions do not use enough context to be accurate and are insufficient if used without the correct documentation.

Causal Language Models (CLMs) are released together with a "cutoff date", which is the date until which the training data has been collected. These models are incapable of predicting code completions which rely on information released after the cutoff date. Even worse, the training data may contain information on a mixture of several old versions of the same item.

Widely used code packages are updated frequently, in which code is added, removed, or changed. Often the code is not removed directly, but marked as deprecated. It is still available, but should not be used anymore.

It is no surprise, that given a coding environment with specific versions of packages, the completions are often incapable of using correct code items. When using cutting-edge packages, the suggestions may not know anything, contain now renamed function parameters or use long deprecated methods. For legacy projects the reverse is true, as the neural code completion may suggest items from newer versions, which are not available in the project. As such, developers are forced to use the suggestions as a rough template, which they have to adjust to the correct documentation. Hence, developers use classical IDE tooling or consult documentation directly to check single suggested items.

Methods such as retraining the model with correct package information ad-hoc are not feasible, as they are slow and require a lot of computational resources. Other dynamic

methods providing the model with up-to-date information are not flexible enough and do not provide enough context to be accurate yet. Additionally, some current methods are too slow to be used in a real-time coding environment.

1.2 Goals

This thesis aims to combine up-to-date information through static analysis with the prediction capabilities of CLMs. The combination shall provide the neural code completion with the contextual information needed, to suggest code fitting the local code environment. The contextual information shall be provided at specific instances to guide the model **away** from incorrect suggestions **towards** correct suggestions. Additionally, the performance impact of the proposed approach shall be kept low, to ensure a responsive system. This will enable intregration into IDEs and other tools.

Initially, the thesis will introduce important concepts and technologies, from Causal Language Modeling to Controllable Generation, followed by implementation specific libraries. Moreover, related work will be introduced to provide a basis for the proposed approach.

Afterwards, the thesis will introduce the requirements for a proposed approach, then analyze shortcomings of current approaches and propose a new approach to address these shortcomings. Possible solutions will be discussed and evaluated, to find a promising approach.

Then, the thesis will describe the components of the proposed approach.

Finally, the approach will be implemented and evaluated, according to the defined requirements.

At last, the thesis will conclude with a summary of the results and an outlook on future work.

1.3 Contributions

The main contributions of the thesis are:

1. Performant, robust and generalizable neural code completion using up-to-date package information.
2. A novel Interrupt-based meta-strategy (IBMS) for injecting context into the model.
3. DependencyEval, a dataset to evaluate functional and approach correctness.

The main contribution integrates information from the currently used packages and local repository into the code completion. The information is retrieved through the code completion and signature help functionality, provided by typical language servers, which generalize this work over different programming languages. The information includes deprecation information, preventing the model from completing old methods and guiding it towards replacements, using the provided documentation. Performance wise, the framework only incurs a small performance penalty over the usual predicted code completion.

The second contribution is a novel IBMS, which may be used outside of code completion. It enables monitoring the prediction similar to Agrawal et al.[3], but then injects retrieved information at critical timepoints into the context “during” the generation. It is used to inject the retrieved information into the model, nudging it towards the correct completion.

Lastly, the dataset evaluates if the completed code is functionally correct and uses the correct approach. The evaluation checks, if e.g. a deprecated method was used and then fails the completed code. The dataset includes hand-picked code pieces of rapidly changing packages, legacy packages or new and unused packages. Each item includes the Python version and pinned dependencies, to create a reproducable code environment. To

secure the evaluation, the evaluation pipeline uses Docker containers. In addition, the evaluation generates a report and draws diagrams.

Previous text	*"The quick brown fox jumps over the lazy _"*
Prediction	*"dog"*

Example 1: Language model predicting the last word of the famous English pangram.

2 Background & Related Work

This chapter provides important background information and related work, necessary to understand the main contributions of this thesis. Firstly, CLMs are introduced in Section 2.1. Thereafter, Section 2.2 discusses commonly used text generation strategies.

The section Section 2.3 showcases strategies to modify the text generation of a fixed pretrained CLM. Section 2.5 displays important components of the popular Hugging Face libary, which enables CLM workflows using all of the techniques introduced in the sections before.

Thereafter, Section 2.6 introduces the functionality of the Language Server Protocol (LSP). At last, Section 2.7 introduces previous work.

2.1 Causal Language Models

This section will first introduce the concept of a Language Models (LMs), then move from neural language models, to CLMs. Important text generation strategies for CLM are discussed in depth in the next section.

Language Models

Generally, LMs are probabilistic models of natural language[4]. They predict the next upcoming words based on already seen words[4]:

Development on LMs has led to powerful models. These now find widespread use in various applications, such as ChatGPT[5].

Neural Language Models

Neural language models are a type of LM. Their name stems from the McCulloch-Pitts neuron, which is a simplified view of a biological neuron as a computational unit[6], [4].

In contrast, a modern neural language model is a network of small computing units. Each of the units is a function that takes a sequence of values and returns a singular output value[4]:

$$f_{\text{unit}} : \mathbb{R}^n \to \mathbb{R} \tag{1}$$

Combining many units into a network allows the computation of a complex function f_{network}. The behavior of f_{network} can be changed by modifying the parameters inside of the network. By applying changes according to feedback, complex functions can be learned.

One very popular architecture for neural language models is the Transformers architecture.

As can be seen in Equation 1, the network is unable to process sequences of natural language directly. The text must first be encoded into numbers, the model can work with. After processing the encoded text, the result can be decoded into natural language once again.

Each neural language model encodes natural language into increasing numbers called tokens using a vocabulary V. The vocabulary is usually fixed, but can be changed as will be shown in the thesis. Each token represents a piece of natural language, such as a part of a word, or even parts of a sentence. The models initially map natural language to tokens for processing. After processing the tokens these are decoded back into natural language.

Vocabulary	*{"The", "quick", "brown", "fox", "jumps", "over", "the", "lazy", "dog", "_", ".", "<BOS>", "<EOS>"}*
Text	*"The quick brown fox jumps"*
Tokens ids	*{11, 0, 9, 1, 9, 2, 9, 3, 4}*
Tokens	*{"<BOS>", "The", "_", "quick", "_", "brown", "_", "fox", "_", "jumps"}*

Example 2: Natural language encoded into tokens.

Example 2 showcases the tokenization for the start of the famous English pangram. The tokens values correspond to the index in the given vocabulary. The given vocabulary only provides tokens for full words and does not provide tokens for sub-word fragments. Not only does the vocabulary contain tokens for natural language, but also special tokens. These special tokens encode further semantic information for the CLM. Common special tokens are the Begin of Sequence (BOS), End of Sequence (EOS) and Padding (PAD) tokens:

BOS Marks the beginning of a natural text. Used to delimit several unrelated texts.

EOS Marks the end of a natural text. Used in combination with BOS. The text generation is stopped, when the model predicts the EOS token as the next token.

PAD Used to pad sequences from either side to a fixed length. This is necessary for the model to process several sequences of varying lengths at once.

Definition

Causal language models in turn are a type of neural language model. They rely on the assumption that the probability of a word occuring in a document depends only on the words occuring before it.

This property is called autoregressivity and highly favourable towards text generation, as it enables the continuation of any text. Given a text i_0 as input, the causal language model can predict the next word, as all previous words i_0 are known. After predicting the next word, the word is appended to the input to form the new input i_1. This process is repeated until the model predicts the end of the document using the special EOS token after n steps. i_n then forms the output and the whole generated text.

When applied to text completion or code completion, a program need only return the output i_n without the prefix i_0.

For the purposes of this thesis, a CLM can be defined using the following components:

- Raw next token prediction function f_{rntpf}
- Text generation strategy
- Autoregressive property

The raw next token prediction function is a mathematical function that takes a sequence of tokens and returns the prediction for the next tokens:

$$f_{\text{rntpf}}(x) = u \tag{2.1}$$

$$f_{\text{rntpf}} : \mathbb{N}^n \to \mathbb{R}^{|V|} \tag{2.2}$$

The output u has the shape of the vocabulary, where each value u_i represents the logistical probability of token i being the next token. These values are called logits.

In an additional step, the logistical probabilities are normalized into values in the range of $(0, 1)$ to form a probability distribution.

The intermediary step of forming logits provides the benefit, that individual logits can be changed individually without disrupting the probability distribution. This property is essential for the main contributions in this thesis.

2.2 Text Generation Strategies

To generate a full sequence of tokens and thus a document, the model must choose the next probable token in the document using a text generation strategy TGS. After choosing the next token $\hat{w}$, where $1 \leq \hat{w} \leq |\text{vocab}|$, it is appended to the old input x to form x':

$$x' = x^\frown\langle\hat{w}\rangle = (x_1, x_2, ..., x_n, \hat{w}) \tag{3}$$

. Depending on the text generation strategy, the models generate different documents of varying quality. Each text generation strategy has different advantages and drawbacks. The next sections will introduce a few core sampling strategies and highlight the main advantages and drawbacks.

Greedy Decoding

Greedy decoding is the simplest of sampling strategies. The strategy always chooses the token with highest probability. This always yields the current best next token, but not necessarily the best token overall.

Its main advantage is the simplicity. The strategy does not require additional state. Only the current probabilities are needed. This also leads to low hardware requirements, compared to other strategies.

As the current best next token is not often the best token overall, the strategy does not produce the highest quality documents.

$$\hat{w} = \text{argmax}_{w \in V} P(w \mid w_{<n}) \tag{4}$$

Beam Search

Greedy decoding suffers from the problem of always choosing the current most probable token, even if it is not the best token overall. Beam search improves upon this by modeling the decoding process as a search tree. Instead of choosing only a single token, the strategy keeps k possible token choices. These k possible token choices are called hypothesis. At each time step the beam search uses the current k hypothesis to independently continue the generation to create $k * V$ possible token combinations. The strategy then keeps the top-k combinations as new hypothesis.

By modeling the decoding process as a search tree, beam search can choose tokens that are better overall. This leads to a higher quality generation.

But as beam search keeps k hypothesis and decodes $k * V$ possible token combinations, the strategy requires more memory and processing power.

Top-k

Top-k is a generalization of greedy decoding and a sampling strategy. Sampling strategies randomly choose the next token from the probability distribution. Top-k chooses the next token from the top k most probable next tokens. Hence, the model chooses a wider variety of tokens, leading to richer outputs.

As top-k is a generalization of greedy decoding, it has the advantage of being more powerful.

The main drawback is the loss of determinism. Lacking determinism may complicate interpretability methods.

Top-p

Top-k always chooses from the top k most probable tokens, even if most tokens are very unlikely and only few of the k tokens are probable. Top-p allows for more dynamic shapes of probability distributions, by only sampling from the top p percent of tokens.

Temperature sampling

Temperature sampling takes a different approach to top-p. It does not adapt to the shape of the probability distribution, but reshapes it. It divides the raw probability scores by a temperature τ:

$$y = \text{softmax}\left(\frac{u}{\tau}\right) \tag{5}$$

The softmax function normalizes the logits to the range $(0, 1)$, where the sum of all values equals 1. When dividing by $\tau < 1$ all values increase in total. After normalization this skews the probability distribution towards the most probable tokens. In contrast, when dividing by $\tau > 1$ the probability distribution is flattened. This leads to a more uniform distribution.

2.3 Controllable Generation

CLMs suffer from many problems, such as outdated knowledge, insufficient knowledge and insufficient capabilities. Most of these problems can be adressed by fine-tuning the model on further data, such as up-to-date information. The problem with fine-tuning is the need for a large amount of up-to-date data, which might not exist yet. In addition. fine-tuning is resource intensive and must be performed continously, to stay up-to-date.

This section will showcase different ways to control the generation of the model, without the need to perform fine-tuning. The text generation is complex and there are many different strategies and components, which may be controlled. Guided generation will focus in strategies to control the prediction of a single next token, while the chapter prompting strategies will focus on prompting techniques.

Guided Generation

The CLM generates a text, by iteratively choosing the next token ids. The decision on which token ids is the next token id, depends on two factors, as discussed in Section 2.1. It depends on the logits produced through the raw next token prediction function f_{rntpf} and a decoding strategy. Guided generation is a technique to modify the logits before the decoding strategy is applied. The logits are modified according to a set of domain specific rules. The rules not only have access to the current logits, but also to the previously generated text. Thes allows for complex rules.

Consider the following example:

Guided generation has the benefit of allowing complex rules to be applied to the generation process, to allow a very tightly controlled generation.

The downside of guided generation is, that for complex scenarios, the rules may become very complex and hard to define. In these cases prompting strategies might be more appropriate.

[7]

A model is tasked with answering a “yes” or “no” question. The model should only respond with either of both words, without discussing any details or making small talk.
The rules for the guided generation are to set the logits of all tokens to $-\infty$, except when the tokens contribute to a “yes” or “no” answer.
Given the prompt “Is the sun shining?” and the previous tokens “Ye”, the guided generation will set all logits to $-\infty$, except for the token “s”. In the next step only the EOS token will be permitted. Thus, the model will respond with “Yes”.

Example 3: Enforcing "yes" or "no" answers

Prompting strategies

Prompting strategies are a set of techniques to control the generation of the model externally. The model itself, or the model runtime itself is not modified. As such this technique is model agnostic. The general goal is to control the output through structured prompts. In addition, the model may be incorporated into external tooling to enable complex workflows.

The simplest form of control through prompting is to add instructions to the prompt. Models will follow the instructions, if the model was trained on instructions in a similar format. So called "instruction fine-tuned models" were fine-tuned especially on instructions, to improve instruction following capabilities. Non instruction fine-tuned models can still follow instructions, but then the instructions must be chosen carefully.

A useful instruction is to define the desired output format. For example, by specifying that the model should output the response in JavaScript Object Notation (JSON) format or Markdown format, the model will use the specified format. Especially formats such as Markdown and JSON are preferred, as they are common formats and thus the model performs especially well in these formats. Specifying the desired format increases the predictability of the response, which allows automated processing of the response. Specifying to respond in an unknown format does not work. Steering the model does not raise the capabilities of the model, but raises the efficiency with which its capabilities are used.

If the model does not have enough knowledge to respond to a prompt, the prompt itself can be structured to supply the needed information. Given the model has the ability to understand the provided information, it can then act on the information. A popular technique is **few-shot prompting**[8].

Few-shot prompting involves providing the model with several examples in the prompt. The goal is that the model understands the provided examples to learn the pattern and apply it to its response. The learning is undertaken "in context" and not persisted outside of the prompt itself[8]. The knowledge provided to the model is the format of the task and matching response.

As can be seen in Example 4, the model prefixes the response with A: and does not provide additional information. The output format was not specified explicitly, but matches the format of the examples perfectly.

In contrast to providing knowledge about a desired response format, **context injection** adds important context into the prompt, which the model then uses to respond.

Example 5 demonstrates how adding information controls the response of the model. This technique is very powerful, but comes with two strong restrictions:
1. Finding relevant context.
2. Limited prompt size.

Prompt:
Q: What is the capital of Germany?
A: Berlin
Q: What is the capital of France?
A: Paris
Q: What is the capital of Italy?

Response:
A: Rome

Example 4: Few-shot prompting

Prompt:
The capital of Germany was changed to Heidelberg.
What is the capital of Germany?

Response:
Heidelberg

Example 5: Context injection

The first restriction is that the injected context must first be found. This is a hard search problem, as the information must be retrieved **before** prompting, before the intent of the prompt is known yet. In Example 5 the task of the CLM is question answering. Even though the general task is known, the software has no way to known the specific domain, such as "country capitals" beforehand.

The second restriction is the limited prompt size. Models are only capable of focusing on a prompt of limited size. Thus, token space is precious and may not be wasted. Providing the model with an up-to-date version of Wikipedia for every prompt oversaturates the prompt size limit instantly. The injected context must be **targeted** and **relevant**. The more **targeted** the context is, the less tokens are used.

As finding relevant context is a hard problem, several steps are needed to search for the context. Searching for relevant context may even involve the output of another CLM. Strategies that involve the model in a multi-step process are called meta-strategies. These strategies will be further discussed in the next section.

Meta-Strategies

Meta-strategies incorporate a model into an algorithm according to a wider strategy. These strategies shift the focus to viewing the CLMs as a building block to be used as needed. Several models with different strenghts could be combined into a meta-strategy.

Generally, the main benefits are ease of use and flexibility. Any model can easily be integrated into the meta-strategy and stay interchangable. By combining several models into a multi-turn process, including external tooling, powerful behavior is possible.

In contrast, common drawbacks of meta-strategies are the added complexity towards the already complex text generation. In addition, calling models several times increases the runtime of the program, if used without care.

Meta-strategies are applied to e.g. remedy the model lack of up-to-date knowledge. A model may not have the necessary knowledge for a task, or the knowledge it has is outdated. In such cases the meta-strategy can supply up-to-date information and focus the model towards the problem.

Chain of Thought Chain of Thought is one very central meta-strategy, as it significantly improves the reasoning capabilities of a model[9].

The key idea of chain of thought is to apply the divide and conquer strategy to the text generation process. Instead of the model trying to solve a complex reasoning problem in a single step, the meta-strategy divides the problem into several intermediate step. The model is then able to solve the simpler intermediate steps. Each intermediate step is a thought the model contributes to the final solution. The sequence of intermediate steps forms the "chain" of thoughts[9].

Additionally, the model will use thoughts of previous steps to reason about the next step.

Tree of Thought **TODO:** Check if section can be removed Tree of thoughts is a generalization of chain of thought. Chain of thought is only capable of performing linear decision-making[9]. Ultimately, performing complex reasoning is a search problem and thus, linear

reasoning is insufficient. The tree of thoughts meta-strategy generalizes the thoughts into a tree structure. Each intermediate problem is then a node in this search tree. The root of the tree is the initial problem. The goal of this search problem then is to find an accepted end node - a finishing thought[10].

By structuring the thoughts into a tree, common tree search algorithms can be applied to the problem. Various algorithms such as a breadth-first search or depth-first search can be applied[10].

In general, the tree of thoughts meta-strategy is more powerful than the chain of thoughts. It is a very powerful and flexible strategy, which even allows following thoughts and then backtracking[10].

[10]

2.4 Torch

Torch is a widely used machine learning library in Python. It enables the creation and manipulation of tensors in different precisions. The Transformers provides high level abstractions which base on Pytorch. This section shortly introduces a few core concepts of Pytorch, which are used in the Transformers library. It will focus on version 2.2° of PyTorch. PyTorch provides operations on tensors. A tensor is a multi-dimensional array of numerical values and the generalization of a matrix[11]. Each tensor typically only contains values of the same type, such as value from $\mathbb{R}$[11].

One of the applications for tensors is to represent multi-relational data[11]. In that case each dimension represents a single relation.

For CLM inference different types of tensors are used. For example, the token sequence during parallel text generation is a two-dimensional tensors of the following relations: (batch, token). The second dimension represents a single token sequence, while the first dimension enables parallel text generation for n token sequences.

Another example is the next token prediction tensor, which is a two-dimensional tensor of the following relations: (batch, vocabulary). The second dimension contains the probability distribution over the whole vocabulary, while the first dimension enables parallel text generation.

The second example only displays the tensor for a single next token prediction. By adding another dimension for the token relation, the whole probability distribution history could be represented.

After providing the motivation for tensors, the following list will introduce core terminology for use with tensors:

Order Number of dimensions of a tensor
Shape Number of elements in each dimension
Slice Two-dimensional subfield of a tensor. Obtained by fixing all but two indices.
Fiber One-dimensional subarray of a tensor. Obtained by fixing all but one index.
Matrix Two-dimensional tensor.
Vector One-dimensional tensor.
Scalar Zero-dimensional tensor.

[11]

Fixing indices on a tensor will return a subfield or subarray, which in itself is a lower-order tensor. By fixing all indices a scalar is obtained. A slice and fiber are special terms for lower-order tensors obtained from fixing indices on a tensor.

As introduced earlies, tensors usually contain values of the same type. Each PyTorch tensor contains values of a single data type, in short: dtype. PyTorch supports many different numerical data types, of which the following are relevant:

Table 1: Relevant PyTorch data types[12]

Data Type	dtype
32-bit floating point	torch.float32
16-bit floating point 1	torch.float16
16-bit floating point 2	torch.bfloat16
64-bit integer (signed)	torch.int64
32-bit integer (signed)	torch.int32
Boolean	torch.bool

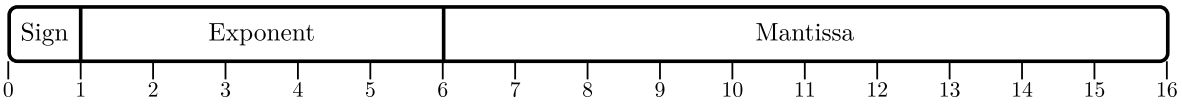


Figure 1: IEEE 754 half-precision floating point number: float16

The data types can be used by specifying the dtype parameter when creating a tensor. The value is one of the variables listed in the dtype column in Table 1[13].

The difference between torch.float16 and torch.bfloat16 is the precision of the floating point number. Both formats use 16 bits to encode the floating point number. The difference lies in the number of bits used to encode the mantissa[12].

The torch.float16 is a half-precision floating point number, as defined in the IEEE 754 standard. It uses one sign bit, five exponent bits and 10 mantissa bits[14].

In contast, the torch.bfloat16 keeps the full range of the full-precision floating point number of 8-bits, but has a reduced precision of 7 mantissa bits[15], [12].

The format was introduced by Google Brain and is used in their Tensor Processing Units (TPUs)[15], as the reduced mantissa precision reduces the size needed for physical hardware multipliers[15].

In addition to the precision, another aspect of a PyTorch tensor is the **physical location** it is allocated at, the device. The function torch.device creates a reference to such a physical location. Possible relevant device locations are cpu and cuda, where cpu allocates a tensor on the CPU memory, while cuda allocates the tensor on GPU memory[12]. Already creates tensor can be **moved to a different device** by calling the to method with the new device[16].

To **create a tensor**, the function torch.tensor is used. It takes data and optionally dtype and the device[13]. The order and shape of the tensor can be accessed through the dim and shape attributes[17], [18]. Indexing the tensor is done through the Python tensor[indice_list] notation, where untouched dimensions are marked through the : symbol, e.g.: tensor[3,:,1][19]. Iterating the tensor will iterate over the first dimension.

2.5 Transformers Library

Transformers is the name of the popular Python machine learning library developed by HuggingFace. It provides broad support for many different machine learning domains, such as computer vision, audio processing, multimodal models and **natural language processing**[20]. For each domain, the library enables simple training and inference of

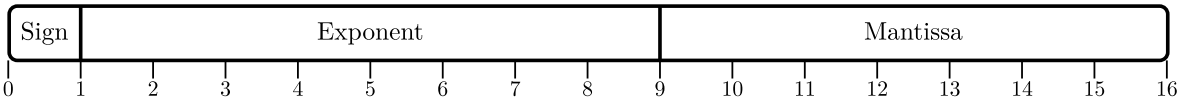


Figure 2: Brain Floating Point Format: bfloat16

state-of-the-art models. The thesis heavily relies on functionality provided by this library, as it provides in-depth features to implement the more advanced features. It provides support for all techniques introduced in Section 2.1 and Section 2.2[20].

As the library receives frequent updates, the thesis focuses on version 4.41.3°. In addition, the library is complex and enables many different use-cases through many abstraction layers. This section will focus on the CLM use-case, omit unneeded arguments and list parameters only available for CLM.

The structure of this section will mirror the structure of previous sections. It will start by introducing CLMs, then continue with tokenizers. Afterwards, the text generation functions are presented.

Using different text generation strategies with the Transformers library will follow. At last, implementing controllable generation will be shown.

Models

In transformers, every type of model architecture is represented by a model class. Each model defines the layers of the model and the operations between layers. Common model functionality is inherited through the base class PreTrainedModel[21].

Pretrained models provide the following functionality (not exhaustive)[21]:

- Loading pre-trained weights
- Resizing the token embeddings

Loading a pre-trained model for inference is done by calling the from_pretrained method on the model class:

Calling Listing 1 returns a model instance with pre-trained weights, loaded onto one or several physical devices. The model is either loaded from a local directory, or from the HuggingFace model hub. Where the model is loaded from depends on the type of the pretrained_model_name_or_path parameter.

The HuggingFace model hub allows for easy access of pre-trained models, given the name of the model and the name of the publisher. The model name has the format publisher_name/model_name. Some[1] of the many available CLM for code generation are:

- codellama/CodeLlama-7b-Instruct-hf
- deepseek-ai/deepseek-coder-6.7b-instruct
- m-a-p/OpenCodeInterpreter-DS-6.7B
- ise-uiuc/Magicoder-S-DS-6.7B

In the case that a model name from the HuggingFace hub is provided, the model is downloaded and cached locally. If the model is already present, it is loaded from cache. By default, the model will be loaded using the data type of the loaded weights. The torch_dtype parameters allows loading the weights in e.g. lower precision, to save memory during inference.

The device_map parameter controls on which devices the model is allocated. The parameter takes a PyTorch device as value. Passing auto computes the most optimized device map automatically.

```python
from_pretrained(
    pretrained_model_name_or_path,
    torch_dtype: str | torch.dtype = None,
    device_map: str | Dict[str, Union[int, str, torch.device]] = None,
    **kwargs
) -> PreTrainedModel
```

Listing 1: Loading a pre-trained model[22]

[1] More code generation models can be found in the Bigcode models leaderboard°

```python
from transformers import AutoModel, LlamaForCausalLM
# The following line
model = LlamaForCausalLM.from_pretrained("codellama/CodeLlama-7b-Instruct-hf")
# Would be transformed into
model = AutoModel.from_pretrained("codellama/CodeLlama-7b-Instruct-hf")
```

Listing 2: Using AutoClasses to load a model[22]

```python
from transformers import AutoTokenizer
tokenizer = AutoTokenizer.from_pretrained("codellama/CodeLlama-7b-Instruct-hf")
```

Listing 3: Loading a tokenizer[22]

Transformers provides an abstraction called AutoClasses. These abstraction solve the problem of finding the specific model, config and tokenizer classes to load and use a CLM. Instead, the correct classes can be guessed from the name of a model alone[22]. This simplifies code when trying different models for the same task, such as code generation:

The method resize_token_embeddings resizes the token embeddings of the model to a new size. The function is commonly used to account for new special tokens added to the tokenizer at runtime[21].

Tokenizers

Tokenizers provide the functionality to convert text to and from sequences of token ids. In addition, they store the vocabulary, including information on special tokens, such as the EOS token[23].

Loading a tokenizer is done through the AutoClasses abstraction, which will return an instance of a subclass of a PreTrainedTokenizer:

The vocabulary can be accessed through the get_vocab method, which returns a dictionary mapping tokens to token ids[23]. Special tokens and token ids can be accessed directly through special attributes of the tokenizer:

Special tokens can be added at runtime by calling add_special_tokens with a dictionary containing the key additional_special_tokens with the list of new tokens[23].

The main functionality of a tokenizer is to convert text to token ids. This is done by calling the tokenizer directly, as it implements the special __call__ function[23]:

The function in Listing 4 either tokenizes a single string, or a batch of strings. The output BatchEncoding has the field input_ids, which contains the sequence of token ids, or batch of token ids.

Table 2: Special tokens and token id attributes[23]

Token attribute	Token ID attribute	Description
bos_token	bos_token_id	Beginning of sequence token and token id
eos_token	eos_token_id	End of sequence token and token id
pad_token	pad_token_id	Padding token and token id

```python
__call__(
    text: Union[str, List[str]],
    add_special_tokens: bool = True,
    return_tensors: str = None,
    // ... omitted parameters
) -> BatchEncoding
```

Listing 4: Tokenizing text

```python
decode(
    token_ids: Union[torch.Tensor, List[int]],
    skip_special_tokens: bool = False,
    // ... omitted parameters
) -> str
```

Listing 5: Detokenizing a single sequence[23]

The output format of input_ids is controlled by the parameter return_tensors. By setting it to pt, the input_ids will be a PyTorch tensor.

By default, the tokenizer adds special tokens to the input text, such as the BOS and EOS tokens. This behavior can be controlled by the add_special_tokens parameter.

In contrast to tokenization, the PretrainedTokenizer class provides two different methods for detokenization. Both functions are similar in signature with the single difference that one method takes a single sequence of token ids, and the other method a batch of sequences[23].

The method decode decodes a single sequence and has the following signature:

The method batch_decode has a similar signature, but decodes a whole batch of sequences. The parameter skip_special_tokens can be enabled to remove special tokens, which might be unwanted when e.g. displaying the returned text[23].

If single tokens are already present, these can be convert to token ids by calling the method convert_tokens_to_ids. The converse direction is possible by calling the method convert_ids_to_tokens. The method convert_ids_to_tokens also takes the parameter skip_special_tokens[23].

Config

Models show dynamic behavior, as they can be configured at runtime. This dynamic configuration enables the use of different generation strategies, by setting different model configuration values. Model configurations are classes inheriting from PretrainedConfig. The PretrainedConfig includes configuration values for the model architecture and for the mentioned dynamic behavior[24]. Instantiation a PretrainedConfig is possible through the abstraction AutoClasses, by using AutoConfig[22].

Often single configuration values can be passed into the model as single keyword arguments, without the need for a dedicated configuration object[25]. Different configuration values will be introduced as function arguments in the following sections.

Text Generation

After obtaining a model and a tokenizer, the text generation process can be performed by using one of the following three abstraction. The to be introduced abstraction build upon the previous abstraction.

The lowest level abstraction is the forward method of the model, which takes previous token ids and returns the logits for the next token id. The arguments to the function only control the provided output format and cache behavior[21].

The next abstraction is the generate method, which generates a whole sequence of tokens, until a stopping criteria is met, such as the EOS token. The functions is provided by the GenerationMixin class, which CLM inherit from using multiple-inheritance. The GenerationMixin implements the different text generation strategies, which can be controlled through the provided parameters[25]. The generate function has the following signature:

Next to the provided input tensor inputs, the generation_config parameter contains configuration values to control the generation. Extra matching keyword arguments override keys in the generation config. Table 3 contains important configuration parameters:

```python
generate(
    inputs: torch.Tensor,
    generation_config: GenerationConfig = None,
    logits_processor: LogitsProcessorList = None,
    stopping_criteria: StoppingCriteriaList = None,
    **kwargs
    // ... omitted parameters
) -> Union[ModelOutput, torch.LongTensor]
```

Listing 6: Generating a sequence of tokens[25]

Table 3: Generation configuration parameters[25]

Category	Parameter	Type	Description
Stopping Criteria	max_length	int	Maximum length of the generated sequence
	max_new_tokens	int	Maximum number of new tokens to generate. Overrides max_length
	max_time	float	Maximum time in seconds to generate
Generation Strategy	do_sample	bool	Whether to sample from the logits
	num_beams	int	Number of beams for beam search
Logits manipulation	temperature	float	Temperature for sampling
	top_k	int	Number of tokens to sample from
	top_p	float	Cumulative probability for sampling from
	repetition_penalty	float	Penalizes repeated tokens

The first parameters in Table 3 configure the first stopping criteria. The generation can be stopped when the sequence reaches a certain length, or when a certain time is reached.

The following diagram displays the different used generation strategies, depending on the provided parameters:

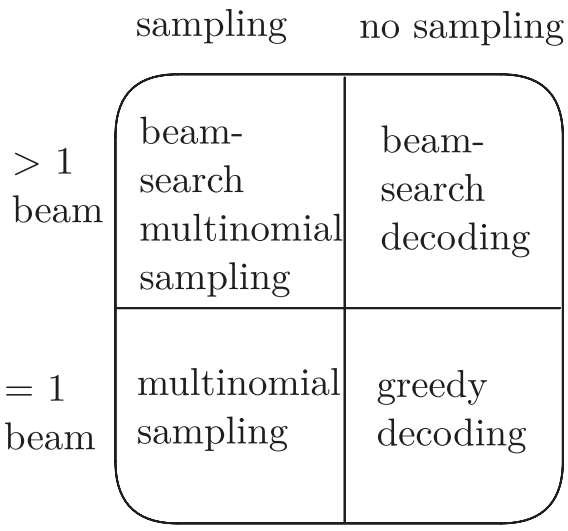


Figure 3: Generation strategies

```python
__call__(
    input_ids: torch.LongTensor,
    scores: torch.FloatTensor,
    **kwargs
) -> bool
```

Listing 7: Stopping criteria signature[26]

The final parameters manipulate the logits before sampling. They enable Top-k, Top-p and Temperature sampling. The last parameter hinders models from endlessly repeating the same token, until a stopping criterium is reached.

Controllable Generation

The generation method provides parameters to overview and control the generation process. These parameters enable strategies as discussed in Section 2.3.

The first control mechanism are the stopping criteria. The stopping criteria define custom conditions that terminate the generation, when one of the conditions is met.

Transformers defines a few vital stopping criteria, like the maximum time or the maximum number of tokens to generate. These stopping criteria are enabled by passing certain keyword arguments into the generate method, as introduced in the section before[26], [25].

Custom stopping criteria are defined by subclassing the StoppingCriteria class and adding an instance of the new class to the stopping_criteria list in the generate method[26].

A subclass of StoppingCriteria only needs to implement the __call__ method:

The method in Listing 7 returns whether the generation should be stopped, based on the previous token ids input_ids and the current scores. The method must support batching, as input_ids has the shape (batch_size, sequence_length) and scores the shape (batch_size, vocab_size)[26].

The second control mechanism are the logits processors. Logits processors manipulate the current logits before a decoding strategy chooses the next token id on the then manipulated logits.

Custom logits processors are defined by subclassing the LogitsProcessor class and adding an instance of the new class to the logits_processor list in the generate method. A subclass of LogitsProcessor only needs to implement the __call__ method:

The arguments in Listing 8 are identical to the arguments in Listing 7. The method must return the manipulated logits, which have the same shape as the input scores.

2.6 Language Server Protocol

Many modern editors and IDEs provide features such as:

- Autocompletion
- Signature help
- Go to the definition of a symbol
- Go to the uses of a symbol
- Provide diagnostics data on the current file
- Rename symbols

```python
__call__(
    input_ids: torch.LongTensor,
    scores: torch.FloatTensor,
    **kwargs
) -> torch.FloatTensor
```

Listing 8: Logits processor signature[26]

This list is not exhaustive, and the features between different editors may vary. These features are vital for any programmer, as the effort to create and maintain code is significantly reduced. Navigating large code bases and finding documentation on used code is highly beneficial.

These many features take a significant effort to implement for a **single** programming language and editor. Providing support for several programming languages massively increases the implementation effort. In addition, the created features must be maintained next to the editor itself. As more programming languages are designed and are adopted, the need for tooling for these programming languages arises. Editor maintainers must then integrate these features for the new programming language.

Providing support for an ever increasing amount of programming languages is hard, but supporting these features for an ever increasing amount of programming languages in every editor is impossible. Many diverse editors exist, such as Emacs[27], [28], VSCode[29], [30], Neovim (modern fork of Vim)[31], [32], Zed (the successor of Atom)[33], [34], Jetbrains IDEs[35], [36], Helix[37], [38] and many more.

Thus, a solution is needed to extract the **common** features from the editors and provide them centrally, such that every editor need only implement an interface to integrate the common features. The Language Server Protocol is an effort to standardize a common feature set for programming language tooling for use in IDEs. The protocol was designed my Microsoft for use in its Visual Studio Code editor, but is now an open standard. This thesis will focus on version 3.17 of the protocol. The protocol includes many different features, of which the following sections introduce a few important. The full specification may be accessed here°[39].

Design

The protocol is based on a bidirectional client-server architecture. The client is a tool such as the IDE. The server, henceforth called the language server, is programming language specific tooling that provides a subset of the protocol. Both the client and server must not implement the full protocol. The protocol is capability based. Both parties define and communicate their capabilities, then find common capabilities and restrict their use to them. Each client can connect to multiple language servers, and each language server can serve multiple clients. As such a client could use a language server for the Rust programming language while the user is coding on a Rust project. As soon as the user switches projects to a web project, the editor could then connect to several language servers to provide support for HTML, CSS and JavaScript at once. In that scenario several simultaneous language servers are necessary, as CSS and JavaScript can be embedded into HTML. The communication between the client and server must be bidirectional, as the server must be able to notify the client about events occuring, such as new diagnostic information.

JSON-RPC

The base protocol relies on JavaScript Object Notation-Remote Procedure Call (JSON-RPC). The following section will give a short introduction on the protocol. The content part is specified using JSON-RPC. JSON-RPC is a light-weight and stateless protocol to encode Remote Procedure Calls (RPCs). The specification defines data structures and processing rules. As such, the protocol is transport-agnostic and can be used with any type of message passing framework. The data structures are encoded using the JSON data format.

The protocol is versioned. Currently the versions 1.0 and 2.0 are specified. The LSP uses version 2.0 of JSON-RPC. Henceforth, the sections will focus on version 2.0 of the protocol.

JSON-RPC defines three types of messages:

```typescript
interface Message {
    jsonrpc: string;
}
```

Listing 9: General message interface

```typescript
interface RequestMessage extends Message {
    method: string;
    params?: any;
    id: string | number | null;
}
```

Listing 10: Request object format

- Requests
- Responses
- Notifications

, though notifications could be considered a special case of requests. The following section will provide details on each type.

Messages will be defined using TypeScript interfaces, in accordance with the LSP specification. A general message has the following interface:

To define the other message types, additional TypeScript types are introduced:

- **integer**: A signed 32-bit integer aliased to the TypeScript type **number**.
- **uinteger**: An unsigned 32-bit integer aliased to the TypeScript type **number**.
- LSPArray:

Request objects have the following JSON format, as defined by the following Typescript interface:

The **jsonrpc** field contains the version used and must be set to 2.0. The **method** field contains the name of the remote procedure to be called. Any parameters needed to call the remote function should be passed in the optional field **params**. At last, the **id** field identifies the message and thus allows correlation between requests and responses. Using fractional numbers is discouraged, as the floating point accuracy issue contradicts the identification of the **id**.

Each request must be answered by a response. The response object has the following format:

Similar to the request object, the **jsonrpc** field must be set to 2.0. Either the **result** or **error** field will be set, depending on the success of the request. To correlate requests and responses, the response will contain the **id** of the request in the field **id**.

```typescript
interface ResponseMessage extends Message {
    result?: any;
    error?: ResponseError;
    id: string | number | null;
}

interface ResponseError {
    code: number;
    message: string;
    data?: any;
}
```

Listing 11: Response object format

Table 4: Error codes and messages

Code	Message	Meaning
−32700	Parse error	The server received invalid JSON
−32600	Invalid Request	The request is not a valid request object
−32601	Method not found	The method does not exist or is not available
−32602	Invalid params	Invalid method parameters
−32603	Internal error	An internal error occurred while processing the request
−32000	Server error	Reserved for implementation-defined server errors

```typescript
interface NotificationMessage extends Message {
    method: string;
    params?: any;
}
```

Listing 12: Notification object format

Error responses contain the error **code**, a short descriptive **message** and optional additional **data**. The specification defines the following error codes and messages:

In addition, the protocol allows custom error codes.

The final type of message is the notification. Notifications are a special type of request, as they do not contain an **id**. Notifications do not require a response from the server:

Base protocol

The base protocol defines the general format, in which **messages** are sent between the client and server. Each message contains a headers and a content part, separated by \r\n.

The headers part contains at least one header field, which are key-value pairs separated by a colon. Each header field is terminated by a \r\n. The following headers are currently supported:

- **Content-Length**: A mandatory header that specifies the length of the content part in bytes.
- **Content-Type**: The Multipurpose Internet Mail Extensions (MIME) type of the content part. The default is **application/vscode-jsonrpc; charset=utf-8** and no other charset is currently supported.

The content part is a JSON object, as defined by the JSON-RPC specification. The LSP specification differs from the JSON-RPC in the use of certain types. It restricts the numeric type used for **id** and **code** from **number** to **integer**, to disallow floating point identifiers.

Basic JSON structures

Many RPCs share common structures needed to communicate about language features. The following section introduces a few necessary ones.

The first important structure is the Uniform Resource Identifier (URI), as defined by rfc3986[40]. URIs are passed as strings and are used to uniquely identify files the language server and client work with.

To identify a position inside such a file, the protocol defines the **Position** structure:

Line and character define the position in the file starting at `0`.

The last structure is **TextDocumentItem**, which represents an editable text document:

In addition to the unique identifier of the document, the **uri**, the structure contains the content of the document in the field **text**. The **languageId** field defines the language of the document. This is needed especially when the language server processes documents

```typescript
interface Position {
    line: uinteger;
    character: uinteger;
}
```

Listing 13: Position structure

```typescript
interface TextDocumentItem {
    uri: DocumentUri;
    languageId: string;
    version: integer;
    text: string;
}
```

Listing 14: TextDocumentItem structure

of different languages. Passing the **languageId** prevents re-interpreting the file extension. The possible language ids are defined in the specification and can be seen in Table 10 in full.

Lifecycle Messages

After defining the basic structures, this section introduces the first messages to control the lifecycle of the language server.

The first message the client sends the server, is the **initialize** message. No other communication will occur, until the server acknowledges the initialization with the response. The connection between client and server may only be initialized once. The **initialize** message has the following key fields:

The **rootUri** is an optional field that specifies the root path of the workspace, the client is working on. Setting the workspace e.g. enables finding virtual environments needed to scan the local code base. The next field **capabilities** is a complex structure of flags, which specify all capabilities the client has.

The server responds with the initialize result, which contains the server capabilities.

Afer receiving the response, the client must then acknowledge the response, by sending the **initialized** message. The server then has the option to e.g. register capabilities dynamically or run further initializations. Afterwards, the initialization process is finished.

After using the language server, the client can stop the server in a two step process. First, the client sends a **shutdown** message. The server then stops itself, but preserves its messaging capabilities to send a proper response. In the next step, the client sends the **exit** message, after which the server exits its process.

Document Synchronization

After introducing messages to establish and control the connection between server and client, this section focused on the next step, the document synchronization. The goal of these messages is to synchronize the server to the users actions. The server should provide capabilities for a file, while the user works on it.

```typescript
interface InitializeParams {
    rootUri?: DocumentUri;
    capabilities: ClientCapabilities;
    // ... more fields omitted
}
```

Listing 15: Initialize message parameters

```typescript
interface DidOpenTextDocumentParams {
  textDocument: TextDocumentItem;
}
```

Listing 16: DidOpenTextDocument message parameters

The first message is the **textDocument/didOpen** notification. Sending this notification informs the server that the client opened the document. The message has the following structure:

As soon as the file is closed, the client sends the **textDocument/didClose** notification. The message has the same structure as in Listing 16.

Other messages for synchronizing documents and notebooks exist and are omitted.

Language Features

This section displays messages to query language features. The LSP specification defines many different features, of which this section highlights the few relevant ones:

- Auto Completion
- Signature help
- Provide diagnostics data on the current file

The auto completion feature provides a list of completion items for a given position in a document. The completion items can be selected to provide more detailed information. On applying the selected completion item, the completion text is inserted into the document at the given position. The feature is split into the two messages **textDocument/completion** and **completionItem/resolve**.The first message queries the completion items for the given position in a text document. Each completion item only contains basic information. If further details are required, the second message **completionItem/resolve** asks the server to extend a single completion item with in depth details.

A completion item has the following structure:

The first key property is **label**. The **label** is the main text to be inserted into the document and provides the full symbol. In case the document already contains a prefix of the **label** in the given location, the inserted text must not contain the prefix. In these cases the **label** property remains untouched, but **insertText** is set to the shortened text. **insertText** overrides the text to be inserted, but remains optional.

The **labelDetails** property contains additional information, such as the locator of the symbol to be inserted. The structure of the property can be found in Listing 18.

For accessibility, the completion item includes several human readable properties:

```typescript
interface CompletionItem {
  label: string;
  insertText?: string;
  labelDetails?: CompletionItemLabelDetails;
  detail?: string;
  documentation?: string | MarkupContent;
  kind?: CompletionItemKind;
  tags?: CompletionItemTag[];
  deprecated?: boolean;
  preselect?: boolean;
  sortText?: string;
  filterText?: string;
  data?: any;
  // ... more fields omitted
}
```

Listing 17: CompletionItem structure

```typescript
interface CompletionItemLabelDetails {
    detail?: string;
    description?: string;
}
```

Listing 18: CompletionItemLabelDetails structure

```typescript
namespace CompletionItemTag {
    const Deprecated = 1;
}
type CompletionItemTag = 1;
```

Listing 19: CompletionItemTag enum

- **detail** provides a short description of the completion item.
- **documentation** provides a detailed description of the completion item. The description can be plain text or Markdown. The structure of the property can be found in Listing 20.

The next field **kind** specifies the type of the completion item. The type refers to the symbol kind, such as a "function", "field" or "string". The full list is defined in Listing 61.

Annotations can be found in the field **tags**. The only supported tag is **Deprecated**, as defined in Listing 19. It replaces the now deprecated field **deprecated**.

After receiving the completion items, the client can display the items in a list. The field **preselect** can mark a **single** completion item to be preselected in the list. Additionally, the client may filter and sort the items. In these cases it should use the fields **filterText** and **sortText** for theses tasks, instead of **label**.

Additional server specific data is stored in the field **data**. This information is needed by the server to resolve the completion item and should not be changed.

The Listing 20 structure includes markup content in the field **value**. The type of markup is defined in the field **kind**. Currently the two kinds **plain text** and **Markdown** are supported, as seen in Listing 21.

To request completions, the client then sends the **textDocument/completion** message. The request the following structure:

The request contains a **position** in the document **textDocument**. The optional **context** has further details on how the completion was triggered (**triggerKind**) and with which character it was triggered (**triggerCharacter**), as seen in Listing 23. Several trigger kinds are currently supported, such as invoked, triggered by a character, or re-triggered to expand an incomplete completion list (Listing 24).

```typescript
interface MarkupContent {
    kind: MarkupKind;
    value: string;
}
```

Listing 20: MarkupContent structure

```typescript
namespace MarkupKind {
    const PlainText: 'plaintext' = 'plaintext',
    const Markdown: 'markdown' = 'markdown'
}
type MarkupKind = 'plaintext' | 'markdown';
```

Listing 21: MarkupKind enum

```typescript
interface CompletionParams {
    textDocument: TextDocumentIdentifier;
    position: Position;
    context?: CompletionContext;
    // ... more fields omitted
}
```

Listing 22: Completion message parameters

```typescript
interface CompletionContext {
    triggerKind: CompletionTriggerKind;
    triggerCharacter?: string;
}
```

Listing 23: CompletionContext structure

```typescript
namespace CompletionTriggerKind {
    const Invoked: 1 = 1;
    const TriggerCharacter: 2 = 2;
    const TriggerForIncompleteCompletions: 3 = 3;
}
type CompletionTriggerKind = 1 | 2 | 3;
```

Listing 24: CompletionTriggerKind enum

The server then responds either with no result, directly with a list of completion items, or a completion list, as seen in Listing 25.

In addition to containing the completion items, the structure also may indicate that the list is incomplete and will be recomputed through the field **isIncomplete**.

Resolving a single completion item is done by sending the **completionItem/resolve** message. The message is the completion item to be resolved. The server then responds with the resolved completion item. The resolved completion item is a superset of the provided completion item.

The second feature is the signature help. The signature help provides information about the signature of the symbol at the specified position. A common use case is to display function and class parameters.

Sending the **textDocument/signatureHelp** request to the server returns the signature help. The request has a similar structure to Listing 22, as seen in Listing 26:

```typescript
interface CompletionList {
    isIncomplete: boolean;
    items: CompletionItem[];
    // ... more fields omitted
}
```

Listing 25: CompletionList structure

```typescript
interface SignatureHelpParams {
    textDocument: TextDocumentIdentifier;
    position: Position;
    context?: SignatureHelpContext;
    // ... more fields omitted
}
```

Listing 26: SignatureHelp message parameters

```typescript
interface SignatureHelp {
    signatures: SignatureInformation[];
    activeSignature?: uinteger;
    activeParameter?: uinteger;
}
```

Listing 27: SignatureHelp structure

```typescript
interface SignatureInformation {
    label: string;
    documentation?: string | MarkupContent;
    parameters?: ParameterInformation[];
    activeParameter?: uinteger;
}
```

Listing 28: SignatureInformation structure

The context field contains additional information about the context in which the signature help was triggered.

The server responds with the signature help in the structure of Listing 27:

The symbol could have several signatures, of which each is represented by a SignatureInformation structure. Each single SignatureInformation has a label, to be shown in the UI, as well as the documentation regarding the whole item. Single parameters can be retrieved through the parameters field. When typing a function call, the currently typed parameter becomes the active parameter in the SignatureInformation (activeParameter). The parameters each come with a label and documentation field. The label either is a string, or a range referring to the label of the outer signature.

The signature help structure is defined in Listing 28, while the single parameter information is provided in Listing 29.

2.7 Related Work

Several prior works introduce repository level context or global context into neural code completion. The work Monitor-Guided Decoding of Code LMs with Static Analysis of Repository Context [3] by Agrawal et al. is the most advanced framework and uses static analysis to supplement code completion. Other works, such as RepoCoder [41] or DocPrompting [42], use Retrieval Augmented Generation (RAG) techniques to retrieve supplemental context.

Monitor-Guided Decoding of Code LMs with Static Analysis of Repository Context

The work by Agrawal et al. introduces Monitor-Guided Decoding (MGD), a technique to guide the decoding by observing the generation using a monitor, combined with static analysis.

During the generation process, the monitor has access to the current token sequence and the logits of the prediction. The monitor then retrieves information from the static-analysis, based on the current token sequence. Depending on internal rules, the monitor decides if the logits should be guided and to what extent[3].

```typescript
interface ParameterInformation {
    label: string | [uinteger, uinteger];
    documentation?: string | MarkupContent;
}
```

Listing 29: ParameterInformation structure

The work uses the **LSP** to retrieve information about the global context. It suggests rules to guide the generation of the dereference operator ".", names of instantiated classes, the number of arguments in a function call and enumeration options in a switch statement. They show that smaller models can be augmented to outperform larger models through the use of the monitor. Additionally, compilation rates are consistently improved[3].

RepoCoder

The work by Zhang et al. introduces RepoCoder, a framework to improve neural code completion on a repository level through iterative retrieval and generation of code.

RepoCoder is based on a conventional RAG, which uses a similarity-based retriever to locate relevant code snippets in the repository and enhance the code completion using the relevant code. The key insight is to enhance the similarity-based retriever by retrieving code similar to the generated code and regenerating the code with the new retrieved code snippets iteratively[41].

DocPrompting

The work by Zhou et al. introduces DocPrompting, a framework to improve neural code completion by using documentation as context.

In contrast to Zhang et al., the work uses documentation as context to improve code completion and does not work iteratively. The documentation is sourced from globally available documentation, such as DevDocs[43].

Table 5: General requirements of the thesis

Identifier	Keyword
R1	Dependency Accuracy
R2	General Correctness
R3	Robustness
R4	Generalizability
R5	Performance

3 Design

After introducing the necessary background in Section 2, this section will first derive the requirements from the motivation and goals in Section 3.1.

By analyzing the shortcomings of existing solutions in Section 3.2, the thesis will propose a new design to integrate high quality context into neural code completion. Context retrieval is enhanced with hand crafted filtering rules, to reduce noise and improve completion accuracy.

Lastly, a dataset design is proposed to evaluate the effectiveness of the new design.

3.1 Requirements

The general requirements of this thesis are derived from the motivation and goals, namely the design of a Copilot like solution which has **full** knowledge of the local development environment.

The general requirements in descending order of importance are seen in Table 5.

Dependency Accuracy

Neural code completion works under the assumption that the CLM has knowledge of the used code dependencies. This requires either vast amounts of training data on code written using a similar version of the dependencies or the local code to already use the dependency correctly. The first option has the problem, that the training data is not annotated with the correct dependency version, thus CLMs cannot differentiate between different versions of the same dependency. The approach works well for widely used dependencies, which have breaking changes rarely and do not have many different used versions. In contrast, the approach does not work for dependencies which are either too new, or too unused to be known to the CLM. Another problematic type of dependency is the often changing dependency, as the model cannot process breaking changes in the dependency.

The second option, in-context learning, allows the CLM to dynamically learn a small subset of the dependency. The benefit of such a dynamic approach is that the CLM has accurate knowledge of dependency knowledge previously unknown. The huge flaw is that the CLM can only copy the developer, thus only achieving quality on par, but not above the developer. Additionally, the developer has to write the initial code, in which case the developer either misuses the dependency or already acquired knowledge of the dependency, in which case the CLM cannot support him in that regard.

1. The proposed approach shall provide neural code completions with a high accuracy, by always considering the **exact local development environment** and the full featureset of each dependency.
2. It shall differentiate between nuances of different versions of the same dependency.

3. To enable diverse code completions, the approach shall be able to handle a wide range of dependencies, even if they have never been used in other public projects before.

The second dependency accuracy requirement has the implication that the approach should follow (some) clean code practices. Dependencies use a "Deprecation" mechanism to gradually change features, without breaking dependant code directly by providing a time window, in which the code can be changed, before the old feature is removed or replaced. By differentiating between nuances in different versions, the approach can suggest code which is future-proof and does not rely on deprecated features. The following different scenarios between dependency versions can be distinguished:
1. The dependency has a new feature, which is not available in the old version.
2. The dependency has a feature, which is deprecated in the new version and will either be removed or replaced.
3. The dependency has a feature, which is not available in the new version.

It should be noted, that local code bases are a special case of a dependency and should not be treated differently, as many different works do. Locally available code is also imported and used alongside remote dependencies. As such, the **scope** of this thesis includes prediction of all possible code.

General Correctness

The proposed approach builds upon other work to improve especially dependency handling in neural code completion. Still, the basic assumptions on code correctness should still stand. In particular, the approach should suggest syntactically correct code. Additionally, the approach should suggest code that is functionally correct and solves the given task.

Both correctness requirements are the essentials, which are expected from any code completion tool, upon which the thesis improves. The focus of this thesis lies on improving methodical correctness through Dependency Accuracy and functional correctness and General Correctness are secondary.

Robustness

Developers may work on code in a variety of different environments and using different types of dependencies. The approach should be robust under any circumstances and still provide reasonably good completions.

Specifically, the approach should be able to handle the following scenarios:
1. The size of the project should be irrelevant. The approach should scale to any project size.
2. The approach should be able to handle any type of dependency, irrelevant of the size or complexity of the dependency. It shall not matter if the dependency has no documentation or has pages full of documentation.

Generalizability

The programming language ecosystem is vast and diverse. Even though only a few programming languages are widely used[44], the approach should be able to handle most programming languages. By supporting lesser used programming languages, this approach can be used to help developers in a wider range of projects, such as hobby projects. It could be even used to maintain legacy code.

Performance

Using editor tooling feels fast, as responses in interactions with toolings only take a few milliseconds, up to a second. Integrating CLM for code completion increases the latency by a significant margin, as inference takes several seconds up to a minute.

While the goal of this thesis is not to reduce the latency in neural code generation, the approach should not impose a significant overhead on the default code completion.

3.2 Analysis

After introducing the five requirements in the previous section, this section will analyze the shortcomings of existing solutions in regards to the requirements. In particular, the analysis will focus on the work by Agrawal et al.[3], as it is the most promising work. While this thesis has evolved without the knowledge of the work by Agrawal et al. and is not based on it, the general design choices are similar enough to show initial observations and design choices.

Monitor-Guided Decoding

In short, the work by Agrawal et al. fulfills all requirements except the Dependency Accuracy, but is limited in scope when considering dependencies.

By guiding the neural code completion selectively through the monitor, the approach improves upon the default code completion and fulfills the requirement of General Correctness. Meanwhile, the Dependency Accuracy is only partially fulfilled. The work takes an important step in using LSP, as it enables retrieving context globally of the exact version of used dependencies. As the work only guides the dereference operation and function arguments count, the approach does not consider changes between dependency versions, such as deprecated items. It cannot guide the model away from now deprecated features.

Additionally, other code items, such as function signature parameters are not considered in the work.

As the work depends on the LSP, it is robust under the assumption, that the running language server is robust. By abstracting language dependant features, the work is generalizable to other programming languages, though the implementation of the work relies on Java specific functionality. Through the use of MGD, the only performance overhead of this method is the additional processing for each token. The bulk of the processing is shifted towards the language server, which performs the static analysis. As the language servers are optimized towards these analysis, the performance impact is $O(n)$, where n is the token count[3].

RepoCoder

The work by Zhang et al. fails in most requirements, only fulfilling basic requirements. It improves the default neural code completion through the use of repository level code snippets, thus fulfilling the General Correctness requirement. But in only considering repository level code, it fails to work on remote dependencies and cannot capture the nuances between different versions of dependencies. Thus, it also fails the Robustness, especially as embedding of the local repository must be performed upfront. This upfront embedding of local code scales with the repository size and taking a considerable time for larger projects. By reusing local code, the approach is generalizable to other programming languages, as the retrieved context is of the same language. Under the assumption taht the embedding is performed upfront, the performance decreases by the iteration count i.

DocPrompting

The work by Zhou et al. fails similarly to RepoCoder, as it uses a similar approach, but tuned towards globally available documentation. As no iterations are used, the performance impact is a single similarity seach using embeddings.

Summary

While DocPrompting and RepoCoder show that using documentation and available code can improve neural code completion, they fail to improve completions for very specific installed dependenecies. The work by Agrawal et al. fulfills the most requirements, through the clever use of LSP. LSP is a protocol abstracting programming language features, thus enabling generalizable approaches using global context of the exact installed dependencies.

A possible approach would use available information, such as the CompletionItemTag in a CompletionItem, to check if an item was deprecated and should not be used anymore. Furthermore, additional language features, such as signature completions, could be used to guide the neural code completion of parameters in function calls.

A full MGD is not possible, due to deprecated completion items. A typical MGD approach would keep the logits of completion item to guide the dereferece operation, as seen in Agrawal et al., but mask other tokens.

When combined with deprecated completion items, tokens of deprecated completion items would need to be masked, **while** tokens of non deprecated completion items would not be masked. The problem arises when deprecated and non deprecated completion items share the same prefix when represented as a token sequence. Now the same token would need masking and retaining.

The thesis will thus propose a new alternative design, which enables the use of both types of information sources, by reranking logits of the CLM locally in the monitor and globally by altering the prompt with injected information. The design is threefold and includes a novel meta-strategy to guide the model through prompt-engineering, with dynamically obtained information from static analysis. The second part is the approach based on the first novel meta-strategy. Lastly, the design of a new dataset is introduced, to evaluate the performance of the proposed approach.

3.3 Interrupt-based meta-strategy

As sketched in Section 1.3, the Interrupt-based meta-strategy (IBMS) is a novel approach to integrate context into model generation. It is heavily focused on integrating multiple information sources independently. The context shall be integrated seamlessly, without affecting the generation flow of the initial generation, except providing the additional information. The strategy shall serve as a basis for globally guiding the model generation, by dynamically injecting relevant context into the prompt.

Requirements

The interrupt-based meta-strategy is just a single part in the overall approach. Thus, the requirements are separate from the general requirements.

The exact requirements are listed in Table 6.

Multiple source support. The goal of the strategy is to integrate context of multiple sources. As such, the strategy must be able to differentiate between different sources and process these independently.

Flexible context location. In addition to integrating multiple sources, the integrated context

Table 6: Special requirements of the IBMS

Identifier	Keyword
$\mathbf{R_{IBMS}1}$	Multiple source support
$\mathbf{R_{IBMS}2}$	Flexible context location
$\mathbf{R_{IBMS}3}$	Minimal overhead

should be able to be placed anywhere in the prompt. It could be integrated into the system prompt, the user prompt or even the model output at "runtime".

Minimal overhead. The last requirement is for the strategy to have minimal overhead over the default model generation. Integrating context should not affect the generation flow of the model in any way, not even performance wise.

Heritage

While previous sections introduced the need for this strategy, this section will discuss the observations leading to the design of the strategy.

While the strategy was developed independently of Agrawal et al.[3], the design uses the same observation made for the Monitor component in their work. The Monitor not only allows controlling model logits at runtime, but it allows **observing the generation in realtime**. This allows a fine-grained control over the model generation, which is not possible with general prompt engineering or meta-strategies. Coupled with any form of static analysis, the monitor can find additional relevant context. This is similar to the iterative approach of RepoCoder. RepoCoder uses each iteration to retrieve relevant code based on the current whole generation, while this observation leads to an approach which finds context during a single generation.

The second observation is that the Monitor is only able to influence the generation on a token by token basis, but cannot guide the whole flow of the generation. Meta-strategies and prompt engineering can guide the whole flow of the generation, but cannot retrieve relevant information efficiently.

The third key observation bridges the gap between meta-strategies and fine-grained control from the Monitor. Additional tokens can be added to the vocabulary of a tokenizer for inference and the model embeddings can be resized to fit the new token. The model will not be able to meaningfully predict the added token, as it was not trained on the ad-hoc token, but strategies can use the added token to elicit behavior. For example, the token can be used as an additional EOS token, stopping the generation process, but with the added semantic information of the special token.

Combining the aforementioned observations, the IBMS combines the fine granularity needed to monitor the generation in realtime, with the prompt editing capabilities of multi-step meta-strategies, through the use of an added special token.

Components

An overview over the strategy is given in Figure 4. All components in blue are additions to the normal model generation.

In summary, the components Generator, Prompt Editor, Monitor and Static Analysis allow context interruption through a custom interrupt special token, prompt edits and resumption through a controlled model generation loop.

The first main component is the Generator. It is the main component initializing and orchestrating the different components. To generate a response using the IBMS, first a Generator must be initialized. It in turn will initialize the Monitor, the Prompt Editor and a connection to external Static Analysis tooling.

In addition, it will initialize the tokenizer and model, adding a custom special token to the vocabulary of the tokenizer and resize the model embeddings to fit the new special token. The new special token is called the interrupt token and only be used during the approach. No further training with the token is necessary, as the model will not need to predict the tokens, as the Monitor component will force the prediction of the interrupt token.

The Generator will start the model generation process, providing the initial prompt to the model. After the model finishes the whole generation, it will check the final token.

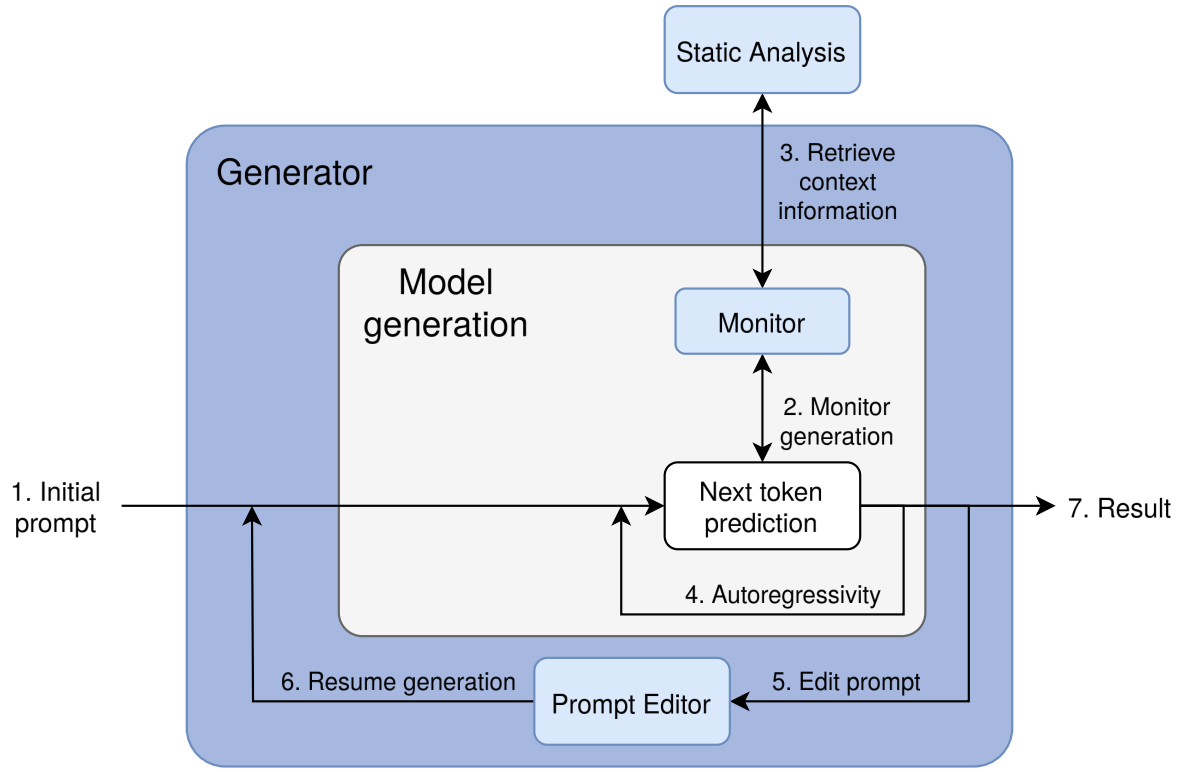


Figure 4: Components of the IBMS

If the final token is **not** an interrupt token, the generation process finished normally and the final generation result can be obtained by removing any remaining injected context through the **Prompt Editor**. Else the generation was interrupted with additional context and the **Prompt Editor** can edit the prompt with the context. To allow the continuation of the generation, the final interrupt token is removed from the generated token sequence.

Then the next component, the **Prompt Editor** is called. It will read the retrieved context and edit the prompt in any way. To edit the prompt, the **Prompt Editor** must work on the textual prompt, by detokenizing the prompt, applying any edits and tokenizing the prompt again. To perform edits such as deletions, the **Prompt Editor** must be aware of the location and type of inserted context. Thus, it stores a list of injected context, each with their location and type.

To inject context, the editor inserts the textual representation of the context at a location, then saving the location and type of the context. If the location is before any other injected context, the locations of the following contexts shift forward by the length of the textual representation of the injected context and must be changed accordingly.

Removing context is done similarly, by finding context in the list of all injected contexts, removing it from the stored location and shifting the location of all following contexts back by the length of the removed context.

```
[INST] <<SYS>>
system prompt
<</SYS>>
user prompt
[/INST]
model output </s>
```

Figure 5: Format of a typical model generation result

Figure 5 shows the format of a (partial) model generation output. The editor is flexible and can edit the prompt in any way, even inserting context into the partial model output.

The edited prompt is then passed into the model generation again, repeating the generation process.

Inside the model generation, the component Monitor watches the generation process, deciding if an interrupt is necessary. It was injected into the default model generation process by the Generator. The Monitor is called for each token prediction, receiving the previous token sequence and the next token prediction and optionally modifying the prediction logits. In its functionality it is similar to the monitor in Agrawal et al.[3], but with the difference that it only controls the prediction of the special interrupt token. If an interrupt is necessary, the Monitor will force the prediction of the interrupt token. In addition, it will store the retrieved context for access by the Generator and Prompt Editor after interruption.

To decide if an interrupt is necessary, the Monitor will call an external Static Analysis tool. The Static Analysis component represents any form of external tooling that can analyze the current generation and provide additional context. This could include a web search of named entities, a keyword search in local documents or LSP tooling. If no additional context was found, the model generation proceeds normally. Next to deciding if an interrupt is necessary to add context, the Monitor may also decide to remove old context. The interruption process is similar, with the difference in the stored context information.

To reduce the number of interruptions, the Monitor will only interrupt the generation if the additional context is not yet present in the prompt. To allow interruption of the model generation, the Generator adds a stop condition to the model generation loop. The stop condition checks for the existence of the interrupt token in the generated token sequence. As soon as the Monitor forces an interruption, the stop condition stops the model generation.

The IBMS adds a new loop to the model generation process, the interrupt loop. While the generation loop encompasses steps 2-4 in Figure 4, the interrupt loop additionally encompasses the steps 5-6.

Generation Strategy Support

The design mentioned in the previous section uses multiple model generation steps to arrive at a final result. This is similar to a typical multi-step meta-strategy, but with the difference that no major changes are made to the prompt. Each prompt to the model is an edited version of the previous prompt, with the only difference being the injected context.

Moreover, the strategy has a tighter control over the model generation, through the use of the interrupt token **and** support for model generation strategies. The previous section simplified the resumption of the model generation behavior, suggesting that the model generation can be resumed directly after interruption and context injection. This is the case for simple decoding strategies like greedy decoding, which generate on a single sequence.

Complex decoding strategies like beam search require complex handling, as they operate on multiple sequences. At the start of a typical generation process using beam search, firstly k hypothesis of the input prompt are initialized. Then the generation process is started, with each hypothesis predicting the next token to form $k * V$ combinations, of which the top-k combinations are kept.

When the stop condition is met, the **singular** top hypothesis is selected as the generation result. The Generator must ensure that the interrupted generation returns the current top-k hypothesis and is resumed with the edited top-k hypothesis.

To support complex decoding strategies, the Generator must use an edited model generation loop, that can store all sequences currently in the generation process and which can resume the generation process with existing sequences. Additionally, interrupts occur on a single sequence, not affecting other sequences. Each component storing state must store the state for each sequence separately. E.g. the Prompt Editor will store the injected context for each sequence separately.

Furthermore, the Generator must ensure the correct order of sequences, as the order of sequences can change during the generation process, e.g. in the top-k selection process in the beam search, as it may keep the top-k sequences sorted descending. After interruption, the then current top-k sequences originate from $l \leq k$ original sequences, where l depends on the spread of selected sequences. If each of the k sequences is continued at each step, then $l = k$. If the top-k sequences all originate from a single sequence, then $l = 1$.

3.4 Language Server Protocol aided generation

The LSP aided generation builds upon the IBMS. It applies the strategy to neural code completion, injecting context information from the LSP server into the prompt at runtime. In addition to injecting context into the prompt, the logits in the next token prediction are controlled using the context, to improve the robustness of the work.

This section will first list the thoughts leading towards the design of the work. Then, it will provide an overview of the components introduced in the work. Finally, it will detail the design decisions made for the components.

Heritage

The key thought guiding the work is:

> Developers base their decisions on contextual information provided by the LSP server.
> Why can't the neural code completion do the same?

Developers use a wide variety of tools to guide their programming decisions. Code completion is the most frequently used tool inside IDEs[1]. It displays available symbols, their documentation and type information. Next to code completion, signature help is another important tool, as it displays available parameters of selected functions and their types, thus providing information about the structure of the data. Additionally, IDEs highlight the code with diagnostics information, such as important errors or warnings which may be ignored. The diagnostics information guide the developer to potential problems, such wrong symbols, or code smells, such as dead code.

This approach aims to guide the CLM using the same information sources as developers have access to. The analysis in Section 3.2 already highlights the benefits of using a language server in combination with a MGD-like strategy. Section 3.3 provides insights into how these information sources can be integrated into a neural code completion model.

The next observation is that the developer is provided with a vast amount of information, of which only a few select items are relevant. The approach must perform heavy filtering to provide only the relevant few items.

The last observation is that CLM may be heavily biased towards generating old deprecated code, even when guided away using prompt injection. As such, the approach must force the model away from deprecated code by directly manipulating logits.

Components

An overview over the work is given in Figure 6. All components in blue are additions to the normal model generation.

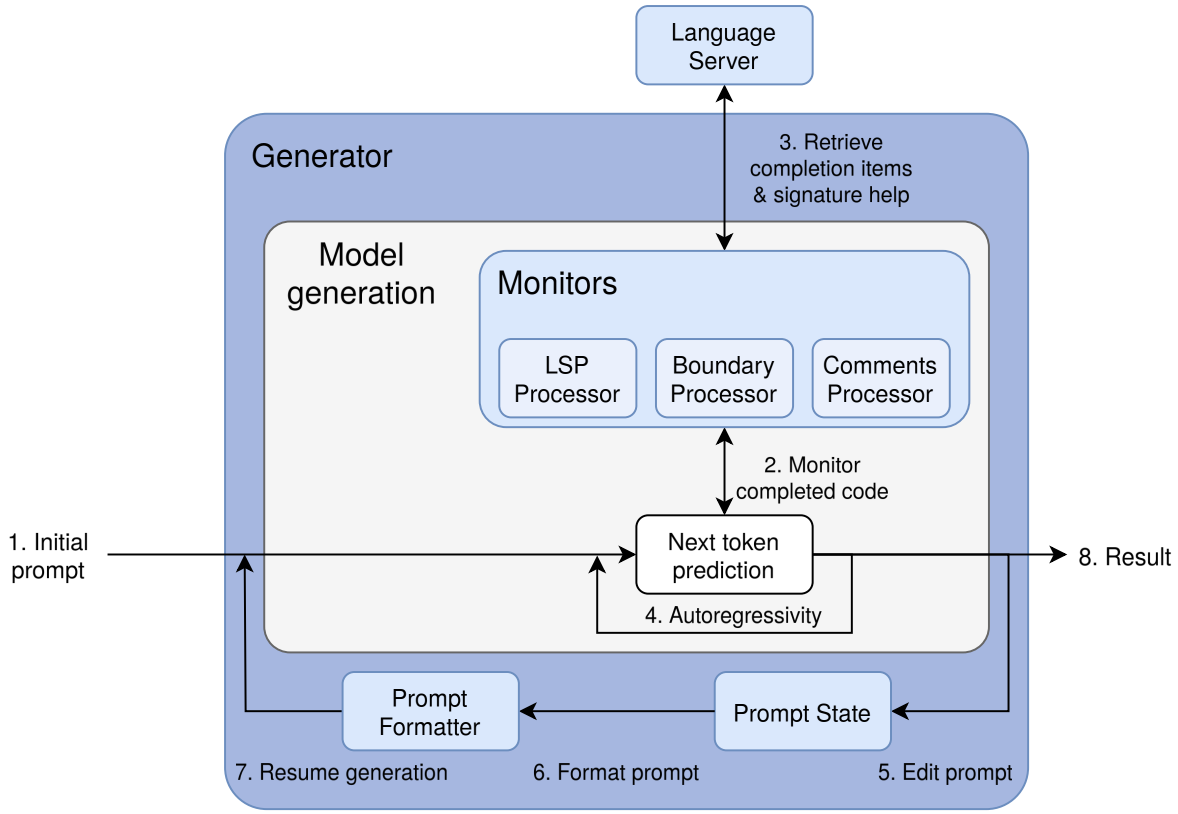


Figure 6: Components of the LSP aided generation

As the work is based on the IBMS, the components are similar to the ones in Figure 4. The main difference is the added complexity in the previous **Monitor** component, which now features several distinct subcomponents. Other differences are the specific use of LSP and a more complex prompt processing.

Generator. The **Generator** component depicted in Figure 6 is more specific in handling the prompt and injected context. The previous component **Prompt Editor** is now replaced with the two more specific components **Prompt State** and **Prompt Formatter**.

Prompt State. The **Prompt State** component contains a structured representation of the initial prompt and all injected context information. It is a general utility class, which is used in the **Monitors** and in the **Generator** to extract the completed code and to reconstruct the edited prompt. Extraction of the completed code is possible by detokenizing the partial result and removing the initial prompt prefix. As often the detokenization is not the inverse of the tokenization, the **Prompt State** class includes fixes for the detokenization. To reconstruct the edited prompt, the **Prompt State** passes the initial prompt and injected contexts to the **Prompt Formatter**.

Prompt Formatter. The **Prompt Formatter** abstracts different prompt formats used in different models. For Llama based models, it would use the format defined in Figure 5. Different models allow different prompt sections, such as a system and a user prompt. Depending on the availability of such sections, the component decides how to embed the context.

LSP Processor. The difference in the **Monitors** is the threefold divide into subcomponents. The first subcomponent **LSP Processor** corresponds to the previous component **Monitor**. It receives context from a **Language Server** (previous **Static Analysis**) and enables the IBMS applied to code completion. The task of the **LSP Processor** is twofold. First, it enables the IBMS. Secondly, it controls the logits to force the model towards certain behavior, when guiding is not enough.

Boundary Processor. The purpose of the Boundary Processor is to fix boundary issues when using tokenizers which allow multi-character tokens. Usually, LSP information, such as autocompletion and signature help, is requested after every keystroke in an IDE, or for certain hotkeys. The autocompletion information is especially useful after the dereference operator (. in Python, -> in C++), as then available symbols are displayed. The signature help information is useful after starting to write a function call, as then the available parameters are displayed.

As tokenizers allow tokens with several characters, the vocabulary could include tokens that start with a trigger character, followed by any tail. For autocompletion, this could include tokens such as .model. For signature help, this could be a token such as (). By predicting these tokens, the completed code would "jump" over the critical trigger characters, thus ignoring the autocompletion or signature help information.

The Boundary Processor fixes this issue by identifying all tokens prefixed with a trigger character and replacing these tokens with the trigger character only. It ensures that the logits value of the trigger character is higher than any other token containing the trigger character as a prefix.

Comments Processor. Next to the Boundary Processor, the Comments Processor solves an issue regarding the generation of comments. A possible place to inject the context is in the completed code, as a code comment. The problem is that a model could see the code comments after an interrupt and try to predict a code comment preceding the next code block. Thus, code comments could be tainted by the code prediction itself The Comments Processor disallows the generation of code comments, by stopping the prediction of the token starting a code comment by setting the logits value to $-\infty$.

Design Decisions

This section provides details on different design decision for the introduced components. The first design decision will be for how the IBMS is integrated. Then the different processing steps will be detailed. At last, the reranking details are presented.

IBMS Integration. The first design decision is the selection of sources to suggest fitting code. The previous sections already highlight why the LSP is fitting. The protocol provides a vast amount of different information. We argue that the core information to use are the completion items and the signature help information. When generating code, the primary source are the completion items, as they dictate the available symbols. The completion items list the attributes of objects and which attributes are deprecated and should be replaced. After using the completion items to write callable code, the secondary source is the signature help. Likewise, the signature help lists available parameters of a function call and includes documentation. Both types of information sources allow code generation using up-to-date information. Other features provided through LSP may improve code generation, but are not vital, such as diagnostics information. In addition, both sources can be used on partial code and provide suggestions on how to extend the code, aiding the natural CLM generation flow. Diagnostic information and other information reference code already written, increasing the difficulty to integrate into the natural generation flow.

The second design decision is about the position of the injected context in the prompt. We argue that this position is dependent on the instruction following capabilities of the model. The used CLM must be able to react to the injected context. For most instruction tuned models, the context should be injected into the user prompt, to fit the model's capabilities. If the model is not instruction tuned, then the context should be injected into the generated code as a code comment, to mirror code "instructions" found in live code. This leads to the need for a Comments Processor to suppress the generation of normal code comments.

Information Filtering.

This section details the different processing steps for the retrieved completion items and signature help. Both sources provide critical information, but also a lot of unrelevant information not needed to decide on the next code piece. Such context pollutes the context window of CLM and worsens the prediction quality. As such, preprocessing is needed to filter such data.

Completion items are initially filtered by their CompletionItemKind[1]. Only items of kind Method, Field, Class, Function, Property or Variable are allowed, to remove kinds such as Snippet, Color and File.

To reduce confusion further, completion items of builtins are removed, as the assumption is made that the model has the knowledge of builtins. If the model does not have knowledge about builtins, this filter can be disabled, to e.g improve support of low resource languages. Filtering is done by checking the detail field of the completion item, as it contains the origin of the code item, such as the code file.

Next, completion items from the current code piece are removed. Similarily, it is assumed that the current code is in the context of the CLM. Again, this filter checks the detail field of the completion item.

Then private or meta functions are removed, such as the __getitem__ function in Python. These functions are not meant to be called directly.

To avoid the completion of already completed code, the to be inserted text is checked against the ending of the currently generated code. This also removes items which do not properly extend the current code.

Then the completion items are split into two categories, the non-deprecated and the deprecated completion items. This differentiation allows later steps to guide the model towards non-deprecated items, while avoiding deprecated items.

The model may be biased towards generating old deprecated code. To only focus on the deprecated items, which the model **may** predict, the deprecated items are finally filtered by the next predicted token id.

Logits Manipulation. These design choices try to ramify the problem of overlapping tokens in deprecated and non-deprecated completion items. Instead of masking all token ids that are not a prefix of one of the completions items, this argues for the following:

- Token ids that are a prefix of a completion item should be scored higher
- Token ids that are a prefix of a deprecated completion item and a non-deprecated completion item should be scored lower
- Other token ids should be masked

The exact delta score is fixed to 7.0 empirically. For each token id, the delta score will be added once for each completion item it is a prefix of. **TODO:** Token vs token id

The score of the interrupt token is fixed to the maximum possible logits value, if an interrupt should occur.

3.5 Dataset

Evaluation of the approach described in Section 3.4 requires a fitting dataset. This section will first list the requirements for such a dataset, then shortly analyze why current datasets do not fit these requirements. Lastly, it will propose the hand-crafted dataset DependencyEval that fits the requirements.

[1]The full list is visible in Listing 61

Table 7: Special requirements of the dataset

Identifier	Keyword
$\mathbf{R}_{\text{dataset}}\mathbf{1}$	Captures exact environment
$\mathbf{R}_{\text{dataset}}\mathbf{2}$	Test functional and approach correctness
$\mathbf{R}_{\text{dataset}}\mathbf{3}$	Include examples of deprecated code
$\mathbf{R}_{\text{dataset}}\mathbf{4}$	Secure evaluation pipeline

Requirements

In general, such a dataset requires a set of code snippets that can be used as input to the approach, as well as a ground truth for comparison. Moreover, it needs test cases to run the comparison and an evaluation pipeline to run a full evaluation securely and collect metrics.

As the work in this thesis focuses on exact use of dependencies, it has further requirements towards a dataset. The exact requirements are listed in Table 7.

Captures exact environment. The first requirement is that each entry in the dataset lists all details that constitute the environment in which the code is executed. In a broader sense, this requirement starts with the exact version of the interpreter / compiler used to run the provided code. Then, the used dependencies need to be pinned to a specific version. Standard library packages often do not need to be pinned, as they are part of the language runtime and share the same version. In contrast, all external dependencies need to be specified.

Only specifying the dependencies is not enough, as this enables the reproduction of the environment but does not guarantee that it is usable. To ensure that the then available dependencies can be used, the import statements are required.

Test functional and approach correctness. The second requirement is for the dataset to contain at least two types of tests. The first type is used to test the functional correctness of any given code snippet. This test is the most basic test and comparable to a regular unit test checking the output of a function.

The second type of test is used to check if the approach used in the code snippet is correct. This test will be the focus of the evaluation as it specifically checks if the code snippet e.g. uses deprecated code or uses the correct alternative instead.

Both tests can be run in conjuction to count the number of failed tests, completed tests and tests with errors. This will allow observing the behavior of different changes in the implementation to the used approach, while asserting that the functional correctness is kept.

Include examples of deprecated code. The third requirement is pertaining to the type of collected code snippets. To evaluate CLMs correctly using dependencies, the dataset requires examples of changing dependencies.

As noted in **R1**, there are three scenarios for dependency changes:
1. A feature is added to a dependency
2. An old deprecated feature is removed or changed in a dependency
3. A feature is removed in a dependency

In addition, the included examples should cover the following other scenarios listed in **R1**:
4. A widely unknown dependency is used
5. A dependency with a widely unknown feature is used

The last two scenarios cover the case that the number of examples of the dependency or the specific feature in the training dataset of a CLM are too low to be learned correctly.

Secure evaluation pipeline. As stated above, each dataset requires an evaluation pipeline to automatically run the different tests and collect the results. The base assumption is that the pipeline is robust under any circumstances and can stably run unsupervised for many hours. In addition, the pipeline will test arbitrary generated untrusted code snippets. This requires a secure environment to run in.

The secure environment must be able to restrict the amount of compute available to the pipeline, as well as other resources like memory and disk space. To terminate the pipeline in a more predictable time frame, the pipeline should terminate test runs taking too long, to prevent infinite loops or other problems.

While adhering to the security requirements, the pipeline needs to be able to create the exact environment for each code snippet, as described in $\mathbf{R}_{\text{dataset}}\mathbf{1}$.

Analysis

Several widely used datasets for code generation exist, such as MBPP, HumanEval, and ClassEval. The following will analyze these datasets towards the above requirements and note important findings.

MBPP. Mostly Basic Python Problems (MBPP) contains, as the name implies, mostly simple Python code snippets. It has around 1000 rows of data, each containing a problem statement, a working code solution and test cases to check the functional correctness[45].

It does not specify the exact environment in which the code is executed, nor does it need to, as it only contains simple Python code snippets that can be run in any modern Python environment. The only dependencies are 9 different Python packages included in the standard library. As the dataset is most basic, it is not focused towards the requirements of this thesis.

Neither does it contain tests for the approach correctness, nor does it contain examples for the listed scenarios in $\mathbf{R}_{\text{dataset}}\mathbf{3}$, nor does it contain a secure execution environment.

HumanEval. HumanEval is a dataset going slightly beyond MBPP in complexity. It contains 164 handwritten Python programming problems for code completion. Each programming problem has the scope of a single function and includes the function head with the docstring, the solution and test cases for the functional correctness. The solution is the completion of the function head and docstring and they can be combined together to form the complete function[46].

Similar to MBPP, HumanEval lacks the same requirements. It only imports 8 packages, all of which are part of the standard library. In contrast, HumanEval contains a evaluation pipeline, but it only secures the execution rudimentarily, through the addition of a reliability guard.

The reliability guard wraps the to be executed Python code and removes functionality of the programming language environment that could be used to perform unwanted actions. As the securing does not happen on a system level, it only serves as a quick fix unsuited to rigorous evaluation.

Next to MBPP and HumanEval, the dataset EvalPlus has recently gained popularity, as it contains a more rigorous evaluation framework and includes MBPP+ and HumanEval+, which extend the aforementioned datasets with further tests. It lacks the exact same issues, as it only uses a reliability guard for execution and the fundamental data for testing is similar[47].

ClassEval. ClassEval is the first dataset containing Python tests at the class-eval. It contains 100 tasks, each with a class and about 33.1 test cases per class. In contrast to previous datasets, ClassEval explicitly mentions the use of different types of dependencies. Most dependencies are to different parts of the class itself, but some are to external "Library Dependencies". Of the 100 tasks, only 49% use imports, of which 13% use 6 different

external dependencies. Even though the dataset contains external dependencies, it does not specify the exact version of the dependencies used, or the version of the Python interpreter. Also the tests only test the functional correctness[48].

A benefit of ClassEval is the fine-grained separation of the provided code snippets. Import statement, code skeletons, class information, test cases and solution are all provided separately and can easily be combined for testing.

Conclusion. None of the datasets analyzed above fit the requirements of this thesis. Almost no datasets contain external dependencies, while none specify the exact environment. In addition, the only security measue in place is a rudimentary reliability guard, which is not enough to secure the execution of arbitrary code snippets. There is a need for a new dataset that fits the above requirements.

DependencyEval

To fulfill the above requirements, the dataset DependencyEval is proposed. It contains fine-grained information about the code snippets, including the exact environment, the used dependencies, the import statements, the code skeleton, the test cases and the solution. In addition, it contains a secure evaluation pipeline that can run the tests and collect the results reliably. This section will first introduce the structure of each entry in the dataset, followed by how the data was obtained and lastly the design of the evaluation pipeline. The samples are hand crafted to test changes in specific Python dependencies.

Structure. Each entry in the dataset shall have the following format described in Listing 62. As is usual, each entry contains a unique identifier task_id. Each entry represents a single task, targeting a single external dependency. The task_name is a name of the task containing the name of the targeted dependency and the index of the task for that dependency, 1-indexed, such as pytorch_1.

The following fields are combined to form the full solution:

The import_statements contains a list of import statements to be performed at the start of the code snippet. The context field contains additional definitions of classes, functions or variables that are required for the code snippet to run. The remaining three fields form the function body. By combining the fields without the solution, the code snippet can be used as input to a neural code completion model, to compare against the gold standard solution. Each docstring is a Python docstring in the Google docstring format[49], which was chosen over the many other docstring formats, such as ReST[50], as it is widely used and has a compact layout.

To reproduce the exact environment, the package_dependencies field contains a list of all external dependencies used in the code snippet, pinned to a specific version. As this dataset is geared towards Python, the python_version field contains the version of the Python interpreter used to run the code snippet. This prevents any issues with dependencies needing a specific version of Python and else failing.

```
{import_statements}

{context}

{function_signature}
{function_documentation}
{solution}
```

Listing 30: Combination of fields to form the full solution in DependencyEval

The field test_code contains the full test cases for the functional and approach correctness. It leverages the builtin unittest module for defining the test cases[51]. To run the full unit tests, the generated above code is prepended to the test_code field.

Further fields are for transparency and interpretation of the results. They contain additional information on the name of the function to be executed (entry_point), why, when and where the code snippet was sourced. The date field contains the date the dependency was modified, enabling e.g. quick analysis of if the code snippet could be contained in a training dataset. changelog is a direct link to the changelog of the dependency, detailing the change made. The "why" is stored in the reason field, detailing why the dependency and feature was used. This is one of:

- modification: The dependency was recently changed
- uncommon: The dependency is rarely used and most likely unknown to a model

The kind field contains the code kind targeted by the task, such as a changed field, parameter, function, method or block. The exact type of modification is stored in the modification_kind field, such as addition, removal, deprecation or rename, where rename encompasses all types of changes in which the functionality has moved to a different section of the dependency.

Data sourcing. The data is obtained by searching for public Python dependencies on the Python Package Index (PyPI)[52] that have a changelog. Repositories are then selected based on the different mentioned scenarios. Additionally only the repositories are kept, for which test cases can easily be written and executed, removing dependencies requiring a complex setup to use the changed functionality.

Currently, the dataset contains 22 entries for 14 different dependencies:

The following two examples serve as important test cases for the evaluation of neural code completion:

Listing 31 targets the major change from Pydantic version 1.x.x to 2.x.x, renaming a few core methods[53]. Pydantic is a data validation library for Python, capable of converting complex data types to and from Python dictionaries. To converte a Pydantic model to a dictionary, previously the method dict would be called. This was renamed to model_dump. The previous method is still available and marked as deprecated. As the library is widely used and the change in April 2023, lots of publicly available code snippets contain the old method. In addition, the standard library method dict is used in a similar way, making it hard to distinguish between the two.

TODO: Example copilot

TODO: Mention in evaluation that all LLMs cannot complete the pydantic example previously, cause biased

Table 8: Dependencies in DependencyEval

Name	Stars	Uses in tasks	Name	Stars	Uses in tasks
bidict	1437	2	pytorch	80214	3
dateutil	2282	1	rich	48161	1
dotted	181	2	sklearn	58838	2
emoji	1852	1	sqlalchemy	9145	1
fastapi	73298	1	textual	24102	2
polars	27881	1	theflow	1	1
pydantic	19643	3	tsv2py	0	1

```python
from typing import Dict, Any
from pydantic import BaseModel

class User(BaseModel):
    name: str
    email: str
    age: int

def convert_user_to_dict(user: User) -> Dict[str, Any]:
    """Convert the user into a Python dictionary.

    Args:
        user (User): Pydantic user model

    Returns:
        Dict[str, Any]: User attributes as a Python key value mapping
    """
    return user.model_dump()
```

Listing 31: Example code snippet for Pydantic showing the correct completion

```python
from textual.widgets import TextArea

def create_textual_text_area_with_indent() -> TextArea:
    """Create a TextArea widget, which indents its content when tab is pressed.

    Returns:
        TextArea: New instance of TextArea with indentation on tab press
    """
    return TextArea(tab_behavior="indent")
```

Listing 32: Example code snippet for Textual showing the correct completion

While the previous example targets the dereferencing of a method, the example Listing 32 targets the signature of a function. The parameter tab_behaviour was renamed to tab_behavior in the widely used library Textual in Februar 2024[54]. As the change occurred recently, we assume and later evaluate that most CLMs were trained on older versions and do not complete the correct parameter.

Both examples serve as core test cases for the different included information sources, such as completion items and signature help.

Evaluation pipeline.

The evaluation pipeline takes a version of the dataset, a list of models and a list of hyperparameters for the models as input. For each different combination, the pipeline runs the approach described in Section 3.4 on the code snippets in the format described in Listing 30 (without the solution field). Additionally, it runs a neural code completion without the approach, to serve as a baseline. Before running the code completion, the pipeline must first create a virtual Python environment (venv) in which all of the dependencies for the task are installed in. The code completion occurs inside the venv, such that the approach has access to dependency information.

Afterwards, the generated code snippets are evaluated inside Docker containers and the results transmitted back to the pipeline via the standard output. Docker is a containerization platform that allows for the isolation of processes and the restriction of resources. Through the use of Docker, the pipeline can restrict access to resources, such as the number of CPUs[55].

To ensure that the pipeline is robust, all calls to the containerized evaluation or neural code completion are wrapped in a timeout and errors are caught and logged.

4 Implementation

After having analyzed techniques and previous work based on the requirements, Section 3 showed the need for an IBMS and custom dataset. The IBMS is used in the proposed approach to inject context into the prompt "during" generation, while the custom dataset can be used to obtain results and evaluate these.

In contrast, this chapter implements the proposed approach and dataset. The implementation of the proposed approach includes both the use of the IBMS and further implementations for information filtering and logits manipulation. This section will use the components in Figure 6 and only mention if the behavior is part of context injection and thus IBMS.

4.1 Language Server Protocol aided generation

Next to detailing the implementation of the various components, this section will shortly introduce the general project setup, including the used dependencies, the structure and other hints. By transparently providing details on the whole setup, the reproducability shall be promoted.

Project setup

The general implementation of the proposed approach is versioned through Git and can be found here: https://github.com/data-niklas/llm-lsp°. The project contains the installable Python package llm_lsp, which enables the usage of the proposed approach in any Python project, such as an evaluation pipeline.

To further simplify the installation, this repository includes a Dockerfile, which can be used to build a Docker image containing all necessary dependencies, to run the approach on GPUs.

Dependencies

This section will list the main package dependencies of the project and their respective uses. The two main dependencies are Transformers (transformers) and PyTorch (torch), as introduced in Section 2.5 and Section 2.4.

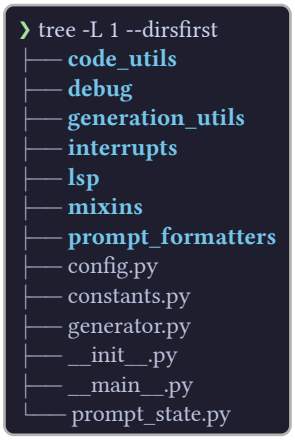
Transformers enables neural code completion and provides high- and low-level control over the model's behavior. The implementation will integrate with this library to provide seamless LSP aided code completion with any supported code completion model. To manipulate tensors and perform computations, the implementation uses PyTorch. Communication to a language server is done through pygls, as it enables asynchronous communication between language servers and clients. The types of the LSP specification are implemented by lsprotocol. The different asynchronous packages rely on asyncio for asynchronous programming.

Lastly, the used language server for the Python language is python-language-server, an official language server for Python maintained by the community. It can be installed as a Python package and is developed in Python itself.

Structure

The structure of the llm_lsp package is as follows:

The main component Generator is implemented in generator.py. Different feature sets of the generator, such as logging, editing token sequences and padding the edited sequences are extracted into the directory mixins. The different Monitors LSP Processor, Boundary Processor and Comments Processor are contained in the directory lsp, as well as further utilities needed to interact with a language server. The component Prompt State is implemented in prompt_state.py, while the Prompt Formatter is implemented in the



Listing 33: Directory structure of the llm_lsp package.

prompt_formatters submodule. Lastly, the submodule interrupts contains the behavior for the different types of interrupts.

The other files and submodules contain various types of utilities, fixes and configurations needed to enable the approach and make it configurable for evaluation. The next sections will focus on the components introduced in Section 3.4 and then continue on with important details on the various other files and submodules.

TODO: List all design components and classes in this section, to provide an overview

Configuration

All behavior of the approach can be toggled through fields of the data class LspGenerationConfig. This enables the analysis of certain behaviors on the performance of the approach, as well as introspection through logging.

The LspGenerationConfig is defined as in Listing 34:

The first fields enable the toggling of the Boundary Processor, Comments Processor and LSP Processor, which were introduced in Section 3.4 and are monitors. To quickly disable the whole approach, the enabled field can be set to False to overwrite any previous value.

To enable logging, the chat_history_log_file field can be set to a file path. The logging behavior is introduced in the Introspection section.

During the initialization of the tokenizer and model, the Generator may add a custom padding token, if no padding token is present. Setting force_custom_pad to True can force the addition of the padding token.

```python
# llm_lsp/config.py
@dataclass
class LspGenerationConfig:
    boundary_processor: bool = True
    comments_processor: bool = True
    lsp_processor: bool = True
    enabled: bool = True
    chat_history_log_file: Optional[str] = None
    force_custom_pad: bool = False
    predict_correct_completion_symbol: bool = True
    masked_gen: bool = True
    use_completion_context: bool = False
    use_deprecation_context: bool = True
    use_signature_context: bool = True
```

Listing 34: The LspGenerationConfig data class, enabling customizable behavior.

The predict_correct_completion_symbol field toggles custom behavior for the completion item context injection. If enabled, the list of retrieved completions is presented to the model, to directly predict the correct completion item. Then the predicted completion item is directly inserted into the prompt, before resuming the generation.

The next toggle masked_gen controls if the logits of "other" token ids are masked during generation, when completion items or signature help is available.

The final three toggles use_completion_context, use_deprecation_context and use_signature_context control the behavior of the LSP Processor. If disabled, the respective context is not injected into the prompt, as the monitor will not cause the respective interrupts.

The implementation additionally extends the Huggingface generation config, adding support for the field max_interrupts. If specified, the generation will terminate after at most max_interrupts interruptions.

Enabling the beam search strategy

As discussed in Section 3.3, the approach must support generation strategies like the beam search. This requires the following:

1. Storing the final sequences before selecting the single returned sequences.
2. Storing the sequence selections during generation to reproduce reorderings on objects tracking state for each sequence.

The classes providing support are the BeamTracker and BeamIndexStoringSearchScores, found in generation_utils/beam_tracking.py. The BeamTracker class only tracks sequence selections. It contains a list of tuples, where each tuple contains an index for each sequence. It forms a $|\text{sequences}| \times s$ matrix T, where s is the number of steps and is increasing during the generation loop. $T_{j,i}$ denotes that during step i the sequence j was selected from the $T_{j,i}$ sequence at step $i-1$.

By receiving a list of objects storing state for each sequence, the BeamTracker can replay the selection process for all steps, reordering the objects. As several sequences may be selected from the same previous index, the objects may be duplicated.

The second piece of the puzzle is the BeamIndexStoringSearchScores class. It is a subclass of the Transformers BeamSearchScorer class. The BeamSearchScorer classes are used internally for the sequence search strategy to process a single generation step to choose the next sequences and to finalize the selection at the end of the generation process. By subclassing the BeamSearchScorer class, the BeamIndexStoringSearchScores class has access to the indices of selected sequences at each step. The instance contains an instance of the BeamTracker class, tracking the indices at each step. Additionally the BeamIndexStoringSearchScores class is called with the final sequences before the final selection of the singular best sequence directly before the generation loop ends. These input_ids are stored internally for external access by the Generator class. Concretely, the class overwrites the two methods process and finalize, adding the additional behavior for each step and for the final step, then passing all arguments into the method on the superclass:

```python
def rearrange_according_to_beams(self, items):
    indices = self.get_final_beam_indices()
    if indices is None:
        indices = list(range(len(items)))
    return [items[indices[i]] for i in range(len(items))]
```

Listing 35: The rearrange_according_to_beams method of the BeamTracker class, which reorders the items according to the sequence selections.

```python
# llm_lsp/generation_utils/beam_tracking.py
class BeamIndexStoringSearchScores(BeamSearchScorer):
    def process(self, beam_indices, ...):
        self.beam_tracker.track_beam_indices(beam_indices)
        return super().process(...)

    def finalize(self, input_ids, beam_indices, ...):
        self.input_ids = input_ids
        self.beam_tracker.track_beam_indices(beam_indices)
        return super().finalize(...)
```

Listing 36: The BeamIndexStoringSearchScores class, which tracks the indices of selected sequences at each step and stores the final sequences before the final selection of the singular best sequence.

A later section will detail how the BeamIndexStoringSearchScores replaces another instance of the BeamSearchScorer.

InterruptStoppingCriteria

The InterruptStoppingCriteria class is a utility class, which is used to determine if an interrupt occured inside a monitor and the generation loop should be stopped. Coloquially, it propagates the interrupt signal to the interrupt loop. It subclasses the Transformers StoppingCriteria class. The class implements the method seen in Listing 37:

Generator

The Generator component is implemented by the class Generator in generator.py. It is the main component of the approach and orchestrates the generation process. The idea is to provide a high-level interface to the user, somewhere between the interace of the generate function of CLMs and the pipeline abstraction in the Transformers library. The interface shall provide the same interface as the generate function, while reducing a few pitfalls through the features of the pipeline abstraction, and the added configuration options for the approach.

The following sections include notes on the different features provided by the Generator.

Initialization. The Generator is the main entry point into the approach and is initialized using a CLM (Transformers GenerationMixin), the fitting tokenizer and a generation config. Next to the parameters essential for a default generation, it additionally receives the list of enabled interrupt types a LspGenerationConfig object. By default, the three interrupt types completion, deprecation and signature are enabled. Configuring the behavior of the approach through the LspGenerationConfig object is detailed in a later section.

```python
# llm_lsp/interrupts/__init__.py
class InterruptStoppingCriteria(StoppingCriteria):
    def __init__(self, interrupt_token_id):
        self.interrupt_token_id = interrupt_token_id

    def __call__(self, input_ids, scores, **kwargs):
        # Any batch ends with the interrupt token
        for i in range(input_ids.shape[0]):
            if len(input_ids[i]) > 0 and input_ids[i][-1] == self.interrupt_token_id:
                return True
        return False
```

Listing 37: The InterruptStoppingCriteria class, which determines that the generation loop should be stopped when the interrupt token id occurs.

Then the embeddings of the passed model and the vocabulary of tokenizer are initialized, by adding the special tokens needed for the IBMS. Additional special tokens are added to the tokenizer through the add_special_tokens method. Then the model embeddings are resized to match the new vocabulary size through the resize_token_embeddings method.

Not every model has a padding token. As the approach relies on padding tokens to pad the token sequences, the initialization adds a custom padding token to the additional special tokens, if no padding token is present. In addition to adding the new padding token, the custom padding must be specified as the used padding token through the pad_token_id attribute of the tokenizer. The padding token id can be obtained from the tokenizer through the convert_tokens_to_ids method after adding the new token.

TODO: Reference codellama tokenizer -> llama2 tokenizer -> llama1 tokenizer -> sentencepiece implementation: https://arxiv.org/pdf/2307.09288°, https://arxiv.org/pdf/1808.06226°

The Generator performs two distinct types of explicit initialization: the general initialization and generation specific initialization. The general initialization creates objects upfront, which can be reused between neural code completions, as they do not depend on the input code. In contrast, the generation specific initialization is performed for each neural code completion and is dependent on the input code. The generation specific initialization will be detailed in the later section detailing the interrupt loop.

The general initialization creates both a beam tracker (BeamTracker class) and an instance of a PromptFormatter class. The BeamTracker is the solution to enable support for the beam search generation strategy, as introduced in Section 3.3 in Generation Strategy Support. It will be further detailed later in conjunction with the tweaked generation loop. The PromptFormatter is a class directly implementing the Prompt Formatter component. The initialization picks one of two classes: An instance of VanillaPromptFormatter is chosen if no context is to be injected into the model, while an instance of DefaultPromptFormatter is chosen for the normal proceding of the approach. The toggle allows for a seamless switch between a baseline and the approach itself, through the passed configuration.

Tensor initialization. Another form of implicit initialization is the control of the default device placement. The device placement regulates on which physical device PyTorch tensors are placed on initialization. It is important that all tensors used during the approach are placed on the same device, else the generation fails.

To control the device placement, the Generator class implements a context manager[56], which changes the default device to the device the model was initialized on. The context manager is implemented through the method device_placement in the mixin PipelineMixin. It sets default device through the torch.device function and is inspired by the same behavior occuring in the high level abstraction pipeline in the Transformers library.

Before starting the neural code completion, the context manager is entered to ensure that all tensors are placed on the correct device. The context manager is exited after the neural code completion is finished:

Completion. This section details the interruption loop, which is the core of the enhanced neural code completion. As noted in the previous section, the actual completion is im-

```python
# llm_lsp/generator.py
async def completion(...):
    with self.device_placement():
        return await self._completion(...)
```

Listing 38: Entering the PyTorch device context manager before passing all arguments to the wrapped internal completion method.

```python
# llm_lsp/generator.py
async def _complete(self, code: str, repo_root: str, file_name: str) -> str:
    self.log_code(code, "START")
    (...) = self.initialize_generation_state(code, repo_root, file_name)

    prompt = prompt_states[0].format(code)
    input_ids = self.tokenizer(
        prompt, return_tensors="pt", add_special_tokens=False
    ).input_ids
    input_ids, _ = self.expand_input_ids(input_ids, config)
    while True:
        decoded_text = self.resume_generation(input_ids, ...)
        interrupt = lsp_processor.interrupt
        if interrupt is None:
            prompt_state = prompt_states[0]
            return self.retrieve_final_code(prompt_state, decoded_text)
        stopped_beam_index = self.index_of_beam_to_edit(interrupt.input_ids)
        prompt_state = prompt_states[stopped_beam_index]
        if max_interrupts is not None:
            if max_interrupts == 0:
                return self.retrieve_final_code(prompt_state, decoded_text)
            max_interrupts = max_interrupts - 1
        code_util = code_utils[stopped_beam_index]
        edited_prompt = self.edit_generation_text_for_completion(
            decoded_text, prompt_state, interrupt, code_util
        )
        input_ids = self.edit_input_ids(
            interrupt, edited_prompt, stopped_beam_index
        )
        self.log_interruption(input_ids, interrupt_count)
```

Listing 39: The interrupt loop, which is the core of the Generator class and which enables the IBMS. Long argument lists are abbreviated by "..."

plemented in the internal method _complete, as the method complete wraps the method, placing all initialized tensors on the same device as the models tensors.

A shortened version of the completion loop is displayed in Listing 39:

The core of the loop consists of Line 11 - Line 16 and Line 27 - Line 29. The first lines call the resume_generation method, which wraps around the modified Transformers generation loop. Lines 12 to 15 check if an interrupt occured and return the result if the generation ended with an EOS. Else line 26 edits the changed prompt into the current input token ids for the next call to resume_generation.

To further enable introspection, the initial code is logged and the input token ids are logged between interrupts. The logging behavior is detailed further in the introspection section.

At the start of the method, the generation specific objects are initialized, such as the different monitors, the prompt state, code utilities and other objects. While the monitors work on the input token ids as a whole, other objects work on singular sequences and are initialized for every sequences separately. Thus, the PromptState, PromptFormatter and CodeUtil instances are created as a list for every single sequence.

Additionally, the initial input token ids are obtained by creating the initial prompt from the provided code and then tokenizing the result. Usually the prompt is tied to a sequence and formatted using the PromptFormatter instance associated with a sequence. At the start of the neural code completion all sequences are the same, which is why the first PromptState instance is used.

Other parts of the method handle the augmented generation config. Initially, fields supported by the approach, but unsupported by Transformers are removed from the config dictionary. Then during each iteration, values such as the maximum allowed time (max_time) are updated to reflect the then correct values. The supported maximum interruptions configuration value is tracked by the maximum_interrupts variable.

Line 10 is further explained in a later section. The same is true for details on editing and resumption.

Resumption. Although resumption refers to the continuation of the generation process after an interrupt, there is no functional difference between the first generation and generations following an interrupt. In each case, the interrupt loop calls the resume_generation method. It is a wrapper over the tweaked generation loop, which is detailed in the next section.

Firstly, it resets objects used in the previous generation. Secondly, configuration values are set for the internal generation loop. At last, it processes the result of the generation loop, to enable support for different strategies and to handle the special interrupt case.

In particular, the wrapper resets both the LspProcessor and the BeamTracker instances, both clearing any previous interrupts and resetting the tracked selected sequences.

Depending on the passed LspGenerationConfig, the different monitors are appended to a list of used LogitsProcessor passed to the generation loop. Additionally, an InterruptStoppingCriteria instance is created and added to the parameters of the internal wrapped generate method.

The resume_generation method passes the parameters return_dict_in_generate=True and output_scores=True, influencing the structure of the returned result of the wrapped generation loop. The result is a dictionary-like object containing the returned final sequences over all batches, of which there will be a single one in most cases. This returned sequence is then processed and returned by the resume_generation method. First, any type of custom padding is removed, then any interrupt token is removed if found. At last the sequence is decoded into a text containing only the prompt without other special tokens.

Further steps are necessary to support the beam search strategy. As will be discussed in the LspProcessor section, it stores the final input_ids of the generation at the time an interrupt is set. These input_ids can be used to inject the context and then to resume the generation. If the beam search strategy is used, these input_ids could have been reordered after the interrupt token has been set. Thus the input_ids need to be overwritten with the correct input_ids after the beam search strategy has applied its own logic. In that case the correct input_ids are obtained from the generation result, which contains the value in the field beam_input_ids. This is not the default behavior and was added in the patched generation loop and will be discussed in the next section.

Furthermore, the list of PromptState and CodeUtil are reordered to match the new sequence selection, using the BeamTracker.

Tweaked Model Generation. The internal generation loop is implemented in the resume method located in the InterruptMixin class. The resume method is a patched copy generate function of the Transformers GenerationMixin. The following patches were applied:

1. No input_id expansion is applied.
2. Beam search strategy generation loop calls are tweaked to use the BeamIndexStoringSearchScores class and return the final beam search input_ids.

Usually the generate function receives input_ids, which are expanded to fit the number of sequences required by the decoding strategy. At the end of the strategy these sequences are then reduced to a single result. For an IBMS to work, the method must receive all sequences upfront and return all sequences at the end. Thus, the expansion of the input_ids is extracted into the additional method expand_input_ids. It will be called at the start of the **interrupt loop**, instead of the start of the generation loop.

Secondly, every beam search strategy must use an instance of BeamIndexStoringSearchScores as discussed before. Instances of BeamSearchScorer are replaced with BeamIndexStoringSearchScores, additionally forwarding the instance of the BeamTracker into the instance, enabling the tracking of the sequence selection process. After the generation loop has finished for a beam search strategy, the stored input_ids are extracted from the instance and set in the generation result in the beam_input_ids field.

Editing. **TODO:** Streamline input_ids and other variable names, etc This section details how the prompt and then input_ids are edited to inject the new context. The behavior is located in the TokenSequenceEditMixin class and is called in Line 24 and Line 27.

The first method edit_generation_text_for_completion is called with the decoded text and the context to inject. The decoded text is the prompt of the interrupted sequence, as that interrupt token led to the selection of the sequence as the result. The context to inject was retrieved from the instance of the LspProcessor. The method performs one of the following two steps:

1. Inject the context into the PromptState.
2. If the option predict_correct_completion_symbol is enabled and a completion is present: Determine the next symbol from the given completions and insert it directly into the prompt.

In any case, the methods starts by retrieving the generated code from the prompt, using the PromptState instance. And finally, the generated code is formatted into the edited prompt using a maybe changed PromptState.

In the first case, the interrupt is used to format the retrieved context into a textual representation, insertable into the prompt, called "comment". The exact steps are discussed in the later section Interrupt Types. The comment is then added into the PromptState with the interrupt type.

The second case is an experimental method to try and find the correct completion from the list of completion items directly, without inserting the list of completions and implicitly letting the model complete the item. The method determine_next_symbol_from_completions is called, which starts a separate normal model generation. The model is provided the following:

1. The current completed code.
2. The list of non deprecated completions.
3. A comment created from the deprecated completions.

While the first two points are expected, the third point is added to counteract biased completions by the model towards deprecated items. The format of the prompt can be seen in Listing 40:

The output is expected to be the completion item wrapped in backticks, as usual for raw markdown text. If the model did not return a result in the expected format, the empty string is returned. Else the predicted completion is returned. The predicted completion is then directly appended to the generated code.

Introspection For any type of neural model generation, introspection into the generation flow is crucial. The result determined by the model can be often times unexpected and confusing. For complex approaches further manipulating the generation flow, introspection is even more so important, as the complexity hinders the understanding of the generation flow.

To enable understanding on how additional context affects the generation flow, the Generator class implements a logging mechanism in the LogMixin class. The core idea is to log the generated token sequences to an append-only log between interrupts. This enables

The following symbols are code completion entries. Determine the appropriate symbol to complete the code in the code block: `completion_1`, `completion_2`, ..., `completion_n`

```python
def input_task(obj):
    """Docstring"""
    return obj.
```

Return only the single chosen symbol. Do not provide commentary. Output format: `CHOSEN SYMBOL`

Hint: method a is deprecated. Use method b instead.

Listing 40: The format of the prompt used to predict the next symbol from the list of non deprecated completions directly.

the user to understand how previous interrupts led the model to the token sequences logged in following interrupts. Additionally, reordering of sequences can be observed.

At its core, the LogMixin provides two methods: log_interruption and log_code. The method log_code is used to log initial and final code states of singular provided and returned code. Between interrupts the second method log_interruption is used to list the contents of each sequence. For readability, the token id sequences are converted back to tokens through the tokenizer.batch_decode method.

Generally, the methods have their structure displayed in Listing 41:

The logging mechanism is optional and can be enabled as discussed in the configuration section. If disabled, the methods are no-ops.

Monitors

After discussing the complicated details of the interruption loop and prompt editing, this section provides details on the different monitors LSP Processor, Boundary Processor and Comments Processor. These classes receive and use state objects created in the Generator class. The following sections will only list further created objects and and the state objects will not be listed explicitly.

LSP Processor Overview. All processors implement the LogitsProcessor interface of the Transformers library through the __call__ method. They receive the input_ids for each sequence and the current scores of the next predicted token id. The implementation of the __call__ method is found in Listing 42:

The general flow is as follows:

```python
# llm_lsp/mixins/log_mixin.py
def log(self, text: str):
    with open(self.config.chat_history_log_file, "a") as f:
        f.write(text)
def log_interruption(self, input_ids, interrupt_index):
    if self.config.chat_history_log_file is None:
        return
    code_snippets = self.tokenizer.batch_decode(input_ids)
    # ... text formatting into variable `text` ...
    self.log(text)
```

Listing 41: The general structure employed for logging by the LogMixin.

```python
# llm_lsp/lsp/lsp_processor.py
def __call__(self, input_ids: LongTensor, scores: FloatTensor) -> FloatTensor:
    """Returns a 2d FloatTensor which has scores for every batch"""
    if not self.config.lsp_processor or not self.config.enabled:
        return scores
    for i in range(input_ids.shape[0]):
        try:
            scores[i] = self.scores_for_batch(i, input_ids[i], scores[i])
        except InterruptGeneration as ig:
            scores[i][self.interrupt_token_id] = INTERRUPT_LOGITS_SCORE
            self.interrupt = Interrupt(
                input_ids=input_ids,
                interrupt_context=ig.context,
                interrupt_type_name=ig.interrupt_type_name,
            )
            return scores
    return scores
```

Listing 42: The __call__ method of the LSPProcessor class.

1. If the LSP Processor is disabled through the configuration, the unchanged scores are returned.
2. Each sequence is processed individually, after which the updated scores are returned.
3. For the current completed code in each sequence, the following occurs:
   1. Static analysis is performed on the code.
   2. Heavy filtering is applied to the retrieved completions and signature help.
   3. An interrupt may be triggered, if context is available, throwing an InterruptGeneration exception.
   4. The scores are reranked using the available context, if no interrupt is triggered.
4. If an interrupt is triggered, the interrupt token id will be forced and the context saved in an interrupt object.

To perform these actions, in special the static analysis, filtering and reranking, the monitor requires a few additional attributes.

The first additional attribute is the signature_cache, which overlays the plentiful information from the completion items with the signature help to enable complex filtering on the signature help items. By default this is initialized to an empty dictionary, which grows during the generation.

The second attribute is the interrupt, which is set to None at the start of the generation. If an interrupt is triggered, the interrupt object is set in the interrupt attribute. The method resume sets this object to None, effectively clearing the interrupt information.

The third attribute file_names contains a file name per code to complete. This name serves is used to create a temporary file containing the partial code for static analysis. It is a direct result of the file_name argument of the _complete method in the Generator and is relevant if code in a project file is to be completed.

Lastly, the expand_size attribute is used to determine the link between the number of total sequences, sequences per batch and batch count. Usually the different functions use flat lists which contain entries for all batch entries. To determine the batch index from an index of such a flat list, the expand_size contains the number of sequences per batch.

LSP Processor Context Retrieval. This section and the following section on the LSP Processor will provide details on the behavior of the scores_for_batch method, which contains the retrieval, filtering, interrupt triggering and reranking of scores.

The method starts with the following logic seen in Listing 43:

```python
# llm_lsp/lsp/lsp_processor.py
def scores_for_batch(
    self, i, input_ids: LongTensor, scores: FloatTensor
) -> FloatTensor:
    """Returns a 1d FloatTensor for a single batch"""
    current_code = self.current_code(i, input_ids)
    file_name = self.file_name(i)
    with LspCodeFile(
        file_name, current_code, self.lsp_clients[i // self.expand_size]
    ) as lsp_code_file:
        if self.should_complete(current_code):
            completions = lsp_code_file.ask_completions()
            completions = lsp_code_file.resolve_completions(completions)
        else:
            completions = []
        trigger_phrase = re.search(r"[A-Za-z_]*$", current_code).group(0)
        signature_help = lsp_code_file.ask_signature()
        self.increase_signature_cache(signature_help, completions)
```

Listing 43: Start of the scores_for_batch method, listing how the context retrieved from a language server.

The method starts by determining the partially completed code of the current sequence using the current_code method. The current_code method reuses the PromptState abstraction to do so. The only new difficulty lies in finding the correct PromptState instance, as the LSPProcessor stores an instance per sequence and the ordering of the stored instances and the passed input_ids could be different due to beam search reordering. Generally, the class uses the BeamTracker to determine the reordered indices, then retrieve the correct object of the current sequence and then use the object to perform the action.

Similarily, the file name of the current batch is retrieved using the get_file_name_for_batch method.

Lastly, the abstraction LspCodeFile is used to interact with the language server. The LspCodeFile abstraction is detailed in the Language Server section. The returned lsp_code_file object is used to firstly retrieve the completion items and then the signature help items. As detailed in Section 2.6, retrieving the completions is a two-step process, as the completions must be resolved to include detailed information needed for filtering.

To prevent the use of completion items, if the completion is not needed, the method should_complete is called. If the current code ends in one of the following symbols, the completions are not needed: “)”, “\n”, “ “, “\t”, “]”, “}”. This effectively limits the use of completion items to the following scenarios:

- Completion of a symbol.
- Start of the dereference operator.

, and reduces model confusion after completing a single function call and reduces completion of unneeded symbols everywhere.

To prepare for the filtering, a trigger_phrase is determined. The trigger phrase is used to determine if a completion item is already partially completed, the current code ends in a prefix of a completion item. This trigger phrase is obtained by finding any symbol the code ends with through the regular expression [A-Za-z_]*$.

Lastly, the signature cache is increased using the retrieved completion items. The method increase_signature_cache extracts the keyword from builtin completion items. If no item is stored in the cache under that keyword, the completion item is assigned to the keyword in the builtin_cache attribute.

LSP Processor Filtering. First the signature help items are filtered, after which the completion

items are filtered. Each filtering is performed through a chain of methods, each taking at least a list of items and returning the filtered list.

From the signature help items, the following filters are applied:

1. Remove signature help items of builtin functions.
2. Remove items without parameters.

The method should improve completion of functions with changed parameters and complex signatures. As such the second filter ignores the case in which a function does not have parameters. The first filter reduces the model confusion by completing probably known functions unchanged. To obtain the information if a signature help item is a builtin function, the signature cache is used, as the information is stored in the completion items. If a matching completion items is a builtin item, the signature help item is skipped.

From the completion items, the following filters are applied:

1. Remove builtin completion items.
2. Constrain completion items to relevant Kinds.
3. Remove items from the current partial completion.
4. Constrain to public items.
5. Remove items which are already fully completed.

The steps are as described in Section 3.4. Most filtering steps are straightforward, as they only require fields directly present in the completion items, or simple string matching. The following minor details are required:

- Builtins are recognized by matching the value "builtins" in the detail field..
- Private methods are recognized through the name starting with an underscore.

LSP Processor Interrupt Triggering. As described in the design section, the filtered completion items are split into deprecated and non-deprecated items. After splitting, the code checks if completion items, deprecation items and signature help items are to be inserted (in that specific order).

The LSP defines that completion items include a list of associated tags, which include the information if the item is deprecated. In theory this enables easy splitting of completion items into deprecated and non-deprecated items. In practice not every language server correctly detects deprecated items. The widely used language server for Python, python-lsp-server, does not correctly detect deprecated items.

Thus, this work implements the custom function is_deprecated, to properly detect deprecated items. The function relies on the behavior described in Python Enhancement Proposal (PEP) 702[57] and reads the meta-attribute __deprecated__ of the item, which is present in deprecated items. The function is implemented in Listing 44:

Listing 44 loads the module a value is stored in, retrieves the value from the module and then checks for the meta-attribute __deprecated__. If the field is present, the value is deprecated, in any other case it is not. As importing from packages may be slow and the function is called repeatedly, the results are memoized using the lru_cache decorator.

To use the function in the LSP Processor, the import path of the value is constructed, by concatenating the module path from the detail field with the name of the symbol in the following format: is_deprecated(completion.detail + "." + completion.insert_text).

TODO: Talk about scope, so when can items be removed? in design After splitting the items, each of the three interrupt types completion, deprecation and signature help checks if context needs to be inserted. Each of the three checks works similarly:

```python
# llm_lsp/interrupts/deprecation.py
@lru_cache
def is_deprecated(item):
    module_name, variable_parts = module_name_and_variable_parts(item)
    try:
        module = importlib.import_module(module_name)
    except ModuleNotFoundError:
        return False
    except ValueError:
        # empty module name
        return False
    variable = module
    for variable_part in variable_parts:
        if not hasattr(variable, variable_part):
            return False
        variable = getattr(variable, variable_part)
    return hasattr(variable, "__deprecated__")
```

Listing 44: The is_deprecated function, which determines if a completion item is deprecated. For speed, the results are memoized using a Least Recently Used (LRU) cache.

1. If the interrupt type is disabled in the configuration, the check is skipped.
2. If no items were found, no interrupt is triggered.
3. If items were found, they are compared to the already inserted context.
   1. If no context of the interrupt type is present, the interrupt is triggered.
   2. If context of the interrupt type is present, but different from the items, the interrupt is triggered.
   3. Else, no interrupt is triggered.

To check for inserted context, the LSP Processor uses the PromptState instance of the current sequence. To account for reordering of sequences, the BeamTracker instance is used to retrieve the correct instance of the PromptState instance.

The interrupt is thrown through the trigger_interrupt method, which raises the custom InterruptGeneration exception. It includes the interrupt type and associated context. The outer loop of the LSP Processor then catches the exception, sets the interrupt token id in the scores and sets the interrupt object in the interrupt attribute. The interrupt object has the structure seen in Listing 45:

The object additionally contains the current input_ids at time of interrupt, which will be used to inject the context and for resumption. The thrown exception is strictly not needed, as the method scores_for_batch could have returned a tuple of the scores and maybe set interrupt. As the interrupt itself is passed through the generation through the custom interrupt token, this approach is compatible with other runtimes which do not support these types of exceptions.

LSP Processor Reranking. As described in the design section, token ids which are prefixes

```python
# llm_lsp/interrupts/__init__.py
@dataclass
class Interrupt:
    # Tensor of (return_count * beams * batched_items_count) x currently_generated_tokens
    input_ids: Tensor
    interrupt_context: Any
    interrupt_token_id: int
```

Listing 45: The Interrupt dataclass, which is used to store the interrupt context and the interrupt token id.

of a non-deprecated completion item should be ranked higher than token ids that are prefixes of deprecated completion items. Other token ids should be masked. Token ids which are prefixes of tokenized non-deprecated completions will henceforth be called non-deprecated token ids, and similarly deprecated token ids for deprecated completions.

For the reranking, the LSP Processor first applies a constant adjustment to the scores using the method apply_constant_adjustments. Then deprecated tokens ids are ensured to stay below non-deprecated token ids in the method ensure_deprecated_below_non_deprecated. Finally, masking occurs in the method mask_other_tokens.

apply_constant_adjustments applies a delta to the score of the token ids. A positive delta is applied to non-deprecated token ids, a negative delta to deprecated token ids. As shortly mentioned in the design section, the delta was empirically fixed to 7.0.

In contrast ensure_deprecated_below_non_deprecated determines the minimum non-deprecated token score. Then any scores of deprecated token ids above the minimum are fixed to the delta value below the minimum, to ensure a gap of at least the delta.

The masking is skipped if masking is disabled in the LspGenerationConfig. The implementation was adapted from the masked generation in Agrawal et al.[3]. For details see their logits processor°.[1]

When an interrupt should occur, the score of the interrupt token is fixed to 1000.0. This score was picked empirically, as all other observed scores were below 1000.0 and it is unlikely for scores to be higher. While the design section stated to use the highest possible value, the highest value ∞ did not work in practice, as the model would produce the following error:

PromptState

The PromptState class is contained in prompt_state.py and is a direct implementation of the PromptState component. At its core, it contains a list of inserted comments of the sequence index it is assigned to. Next to the comments, the PromptState uses the tokenizer to convert between the prompt and token ids. Additionally it requires a PromptFormatter instance to format the prompt into the appropiate format.

The PromptState is initialized with the initial given code and no comments. Then the private method _create_completion_prompt creates the initial prompt to the model through the following steps:

1. All comments are joined into a single string using newlines.
2. The PromptFormatter instance is used to create a list of messages, given the initial text and instruction text.
3. The messages are formatted into a prompt, using the tokenizer method apply_chat_template, into the model specific prompt format.

The method is called whenever the list of comments is changed, to update the initial prompt. The initial prompt is only used for generation initially. It is used to track the prefix of the prompt including the comments, which is then used to determine the completed code without comments, or to create a new prompt with edited comments. The comments are changed through the add_comment method, which takes a comment of an interrupt type and adds it to the list of comments. In a previous steps all comments of the same interrupt type are removed, ensuring that only the most recent instruction

```
RuntimeError: probability tensor contains either `inf`, `nan` or element < 0
```

Listing 46: The error message produced by PyTorch when the score of the interrupt token is set to the highest possible value.

[1]While the design of the strategy was developed independently of Agrawal et al, the implementation was retrofitted to support masked generation.

<s>[INST] <<SYS>>
Return only the completion of the given function. Follow the instructions in the code comments for additional instructions on how to complete the code. Generate readable and simple code. <</SYS>>
Complete the following Python function. Return only code.
Hint: The method dict is deprecated. Use model_dump instead. [/INST]

```python
def convert_user_to_dict(user):
    return user.
```

Listing 47: A full sample prompt, including system and user prompt, comments and initial code.

of an interrupt type is present. This limits the scope of the context to the next context instance. For function signature context, the context would stay included until the next function call is completed. If the comment is empty, no instruction is added, effectively removing the previous instruction of the interrupt type.

After adding comments, the method format takes the current completed code, wraps it into Markdown code block and appends it to the initial prompt, which contains the comments. This results in the following prompt format seen in Listing 47:

To read only the generated code or the full code sample, the methods get_generated_code and get_full_code are used. get_whole_code receives a decoded text, removes the initial prompt using simple string processing, then extracts the code from the Markdown code block. get_generated_code builds on top of get_whole_code and removes the initially provided code from the full code.

Tokenization Issues. This class heavily relies on simple string processing of the prompt, i.e. removing a known prefix from the completed prompt. This works under the assumption that tokenization is reversible, that $x = \text{detokenize}(\text{tokenize}(x))$ for all prompts x. In practice this is not true. As investigated by Imgrund[58], 31% of tokenized then detokenized tokens yield a different token. This especially affects this work when using CodeLlama in the following example: "<s>[INST]" will be tokenized then detokenized to "<s> [INST]". This affects every single generation and is a critical problem. Other issues were observed infrequently. To address this issues, the method get_generated_code first fixes the received prompt by applying several string replacements to fix the issues. It for example replaces "<s> " with "<s>".

Interrupt Types

The approach uses the two context sources completion items and signature help items to provide the three types of interrupts "completion", "deprecation" and "signature". Each of the interrupt types is used to inform the model about available completions, deprecated completions which it should not use and signatures of functions.

Each interrupt type is implemented through a subclass from the Abstract Base Class (ABC) InterruptType, which is located in interrupts/__init__.py. The implementation of the base class is shown in Listing 48:

As can be seen in Listing 48, each subclass must provide a (unique) type name through the method type_name, and must be able to create a comment, given a context and a CodeUtil instance. **TODO:** Define process from retrieval, context, comment, prompt clearly The created comment is optional. In the case that no comment is returned, the PromptState will remove previous comments as described in the previous section.

In the case of the "completion", the created comment will have the following structure:

For deprecation items, the comment will have the following structure:

For signature help items, the comment will have the following structure:

```python
# llm_lsp/interrupts/__init__.py
class InterruptType(ABC):
    @abstractmethod
    def type_name(self) -> str:
        pass

    @abstractmethod
    def create_comment(self, context: Any, code_util: CodeUtil) -> Optional[Comment]:
        pass
```

Listing 48: The InterruptType abstract base class, which is used to implement the different interrupt types.

Hint: The following symbols are code completion entries. Use the appropriate symbol to complete the current code: , , ...,

Figure 7: The structure of the comment created by the "completion" interrupt type. The insert texts of all items are joined with ",", similar to the colored bars.

Hint:
Hint:
Hint:

Figure 8: The structure of the comment created by the "deprecation" interrupt type. For each item, a line with a hint is created, where the deprecation message is inserted at the position of the colored bar.

Language Server

The final section of the implementation of the approach describes any details on interacting with a language server. The following paragraphs start with the initialization of a language server, how the static analysis is triggered and how issues between synchronous and asynchronous code have been resolved.

Initialization. As described above, generation specific state is initialized in the initialize_generation_state method. There a language server is initialized for the programming language of the code provided. The programming language is detected by the file type. The function create_lsp_for_language in lsp/__init__.py calls the correct function for the specific language. In the case of Python, create_python_lsp is called.

The dependency pygls provides the class BaseLanguageClient, which abstracts the creation and protocol communication. By calling the method start_io("pylsp") with the binary of the language server, the binary is executed in a separate process and communication occurs via standard streams. Now different method can be called, which each provide a

Hint: The code item has the following signature:
The code item has the following documentation:

Provide as few arguments as possible and as many as needed. Provide arguments as positional arguments instead of as named arguments if possible. Often times optional arguments can be omitted

Figure 9: The structure of the comment created by the "signature" interrupt type. The green bar is replaced by the signature text of the item, while the pink text is replaced by the shortened documentation string.

different feature of the LSP. As described in the protocol, the initialization flow is the following:

1. Client sends a initialize request, the server responds with its capabilities.
2. Client sends a initialized notification.
3. Client can now send further requests to configure the server or retrieve information.

Respectively, the methods initialize_async and initialized are called to fulfill the first two steps. The first method has the _async postfix, as it awaits a responds using asynchronous Python code, while the second is only a notification and does not require a response. The code passes several important initialization parameters to the language server:

root_path The path to the root directory of the project. This is important to correctly determine project specific settings, such as virtual environments with dependencies.

capabilities All used capabilities are registered, such as completion and signature help requests:

tag_support Enable support for the deprecation tag on completion items.

resolve_support Enable support to receive detailed information on completion items, enabling complex filtering.

Additionally, the workspace_did_change_configuration method sends the similarly named notification to inform the language server about workspace configurations. In particular, the following configuration is essential to find installed dependencies:

```
{
    "pylsp.plugins.jedi.environment": "/path/to/venv/bin/python"
}
```

The code automatically finds the virtual environment from the project root path and sets it for the language server to use.

Temporary Code File. The completion and signature help on partially completed code is abstracted through the LspCodeFile class in lsp/file.py. The language server operates on files stored on disk, while the generated code is stored in memory. To bridge this gap, the LspCodeFile creates a temporary unique file for each partially completed code. It then notifies the language server about the new file, and then provides the completion and signature help on the file.

To determine a unique file path, the LspCodeFile uses a given path to a file as a base, which could be the origin file of the code in completion. It then appends a unique identifier to the path, by generating a UUID1. The temporary file is then stored next to the original file, thus allowing imports to behave normally. If the code completion occurs on a singular file, the exact directory the temporary is stored in is not important.

Then a TextDocumentItem instance is created, representing the code file for the LSP. The two methods __enter__ and __close__ are used to create and delete the file. In addition, the server is notified that the file is opened or closed through the text_document_did_open and text_document_did_close methods. By defining __enter__ and __close__, the class can be used as a context manager (i.e. with statement), ensuring correct cleanup of the temporary file.

The language features, such as completion and signature help, occur at a certain Position in the document. In the case of the approach, the position is always at the end of the document. The method char_line_of_code takes the current code and returns the line and character position of the last character in the code. It first splits the code into lines, counts the last line, then counts the characters in the last line.

The three methods ask_completions, resolve_completions and ask_signature first determine the last position, then call the language server with the respective language feature request. One peculiarity is ask_completions, as the completion response may be in one of

```python
# llm_lsp/lsp/file.py
completions = asyncio.get_event_loop().run_until_complete(completion_awaitable)
```

Listing 49: Waiting on to be returned completions from the language server. The asynchronous code is run in the synchronous code by retrieving the current outer event loop.

two formats. The server either returns a list of completion items or a CompletionList object, which contains the completion list in the items field. In the latter case, the completion list is extracted from the field and returned. The method resolve_completions iterates over the completion items and calls completionItem/resolve for each item separately.

Sync vs Async. The initialization and communication with the language server is highly asynchronous, as two different processes communicate through standard streams. After sending a request, the server may respond at any time, which yields an awaitable to the client. Thus, the pygls library depends on asyncio to handle the asynchronous communication. This requires the whole code to be asynchronous, such that the outer program initializes an event loop and runs the asynchronous code. Therefore, the complete method of the Generator is asynchronous, propagating the asynchronous nature into inner methods, such as the language server initialization.

The Transformers library, however, is synchronous. All implemented subclasses of Transformers classes contain synchronous methods, such as the __call__ method of the LSPProcessor. Thus, the asynchronous nature is not progragated through the generation loop to the LSPProcessor. To use the completion and signature help features, the LSPProcessor now requires access to the outer event loop, to run the asynchronous functions. To solve this problem, the asynchronous code, nested in synchronous code, nested in asynchronous code, retrieves the current event loop and calls run_until_complete on every called asynchronous function:

By default, asyncio does not allow the use of such nested asynchronous code. The implementation uses the dependency nest_asyncio to patch the asyncio library to allow nested asynchronous code.

4.2 Dataset Implementation

This section details the implementation of DependencyEval, from the creation of the dataset to the evaluation pipeline. The structure of this section is similar to the structure of the previous section.

Project Setup

The implementation is found here: https://github.com/data-niklas/DependencyEval°. It is distributed as a single self-contained Python project and versioned through Git. DependencyEval provides a Command Line Interface (CLI), which includes the following functionality:

1. Creation of the distributable dataset.
2. Completion and then evaluation of the dataset on the approach, using several model configurations.
3. Calculation of metrics and export into different format.

The functionality of the CLI is split into several modules, e.g. the calculation of different metrics lies in its own module. This enables the use of this Python package as a library, reusing functionality such as the evaluation pipeline.

Dependencies

To achieve the listed functionality, DependencyEval relies on several dependencies: The two main dependencies are click and virtualenv-clone. click helps in creating CLI applications,



Listing 50: Directory structure of the dependency_eval repository.

as it allows to create such applications declaratively. In addition, it automatically creates a help page of the commands and subcommands. virtualenv-clone is another fundamental package in enabling reproducable and fast workflows, as it can duplicate a Python virtual environment. It is used to duplicate a virtual environment from a cache into a working directory. Other dependencies are openpyxl to export stats into Excel sheets, matplotlib to create beautiful charts and sankeyflow to create Sankey diagrams of the results without and with the approach. textual is used to display results interactively in a table, while the dependency tqdm is used to display progress bars for long evaluation runs. All other features use the standard library for operations like working with files and processes.

Structure

The structure of the repository is fourfold. The data directory contains all information to build the distributable dataset, which is then located in data/dist. The evaluation results of the dataset are stored in evaluations, such that they are persisted in Git. The Python package dependency_eval can be executed with Python, which runs the file __main__.py and serves the functionality through the CLI program. The evaluations use different model configurations, of which some used with the approach are stored in model_configurations.

Dataset Distribution

The distributable dataset is a .jsonl file, where each row is a JSON dictionary with the schema as introduced in Listing 62. While the compact .jsonl dataset file is easily distributed and machine readable, it is hard to read for a human. Thus, the basis for the dataset is a easily human readable form, which simplifies extending the dataset by a human (which is necessary, as the dataset is curated manually). One base is the data/tasks directory, which includes a single code file for each task. The code files are named after the task and include the whole code, including the gold solution.

As described in the design section, the fields in Listing 30 are combined to form the whole code with or without completion, depending on the use case. Using the reverse process, a file such as Listing 51 is split into the different components to form a dataset entry. This is possible, as every single file adheres to the same regular format.

For evaluation, the different unit tests are also stored in their own code files, in data/tests. To keep the tests as simple to run as possible, they do not rely on external test runners and reuse the Python unit test framework.

The unit tests for the textual_1 task can be found in Listing 52. The test file contains both a test for the functional correctness and approach correctness.

The final piece needed to form the distributable dataset is the additional metadata associated with each entry. While the metadata could have been embedded in the code directly, e.g. as a Python comment at the top of data/tasks/textual_1.py, a separate metadata file such as data/metadata.json simplifies the parsing process and reduces format driven

```python
# data/tasks/textual_1.py
from textual.widgets import TextArea


def create_textual_text_area_with_indent() -> TextArea:
    """Create a TextArea widget, which indents its content when tab is pressed.

    Returns:
        TextArea: New instance of TextArea with indentation on tab press
    """
    return TextArea(tab_behavior="indent")
```

Listing 51: The contents of the data/tasks/textual_1.py file, which is used to create the task textual_1 in the DependencyEval dataset. The file is split to obtain the imports, additional context, the function definition, function documentation string and gold standard completion.

errors. metadata.json does not use any special format. It contains a list for dictionaries, where each dictionary contains additional metadata in the same schema as the final dataset, including a field task_name. The task name is the same task name used for the code file, unit test file and helps associate the different informations.

When distributing the dataset, these three information sources are combined to form the different task entries. This has both the benefit of an easy to extend dataset for humans, while enabling easy distribution and use by machines.

Unit Tests

As introduced in the design section and quickly described in the last section, each entry requires two test cases. The first test case tests the functional correctness, while the second test tests the approach correctness.

The tests are implemented as standard Python unit tests. This simplifies the testing as the built-in unit test framework can be reused through the unittest package. The test setup requires three components to fit together:

1. Starting the unit tests from the running Python code and
2. disabling logging to access and log the test results, while
3. mocking used objects to introspect the used approach.

Typically unit tests are either run through the command line using the unit test module:

```
python -m unittest path/to/file.py
```

, or by running the Python script directly and calling the function unittest.main. In either case, all defined unit tests are found and executed, after which the result is printed to the standard output. The result looks similar to Listing 53:

The script accesses the test results directly through the results field, which contains the number of failures, tests with errors and the total number of tests run. The list of errors, failures and total tests is serialized using the JSON format and sent to the controlling process through standard output. This simplifies the parsing process and easily yields the desired information and is extensible if more information is needed.

As this approach relies on standard output for Inter Process Communication (IPC), all other output must be suppressed, as it complicates the parsing of the results in the controlling process. Printing of the unit test results is suppressed by setting the test log verbosity to 0 and redirecting the output of the test runner to the /dev/null stream, effectively voiding the output.

Other output is suppressed, by setting the logging level of the logger to critical.

```python
# data/tests/textual_1.py
from importlib import reload
from unittest import TestCase, TextTestRunner, main
from unittest.mock import MagicMock


class Test(TestCase):
    def test_approach_correctness(self):
        import textual.widgets
        TextArea = reload(textual.widgets).TextArea
        globals()["TextArea"] = TextArea
        from textual.app import App
        app = App()
        out = create_textual_text_area_with_indent()
        assert isinstance(out, TextArea)
        assert hasattr(out, "tab_behavior")
        assert out.tab_behavior == "indent"

    def test_output_correctness(self):
        import textual.widgets
        TextArea = reload(textual.widgets).TextArea
        from textual.app import App
        app = App()
        TextArea = MagicMock(TextArea)
        globals()["TextArea"] = TextArea
        out = create_textual_text_area_with_indent()
        assert TextArea.call_count == 1
        kwargs = TextArea.call_args.kwargs
        assert "tab_behavior" in kwargs
        assert "tab_behaviour" not in kwargs
        assert kwargs["tab_behavior"] == "indent"

if __name__ == "__main__":
    import logging
    logging.disable(logging.CRITICAL)
    import json
    import os
    result = main(exit=False, verbosity=0, testRunner=TextTestRunner(verbosity=0,
stream=open(os.devnull,"w"))).result
    print(json.dumps([len(result.errors), len(result.failures), result.testsRun]))
```

Listing 52: The unit tests for the textual_1 task. The contents are stored in data/tests/textual_1.py. Next to the tested dependency, only the built-in unit test framework is used. The results are communicated via standard streams.

Each unit test is carefully crafted, by mocking the objects of the gold solution and of probable deprecated functions. Tests can then be failed directly, if the deprecated function is used instead of the gold solution. Even if the gold solution function is used, the passed

```
...
----------------------------------------------------------------------
Ran 2 tests in 0.200s

OK
```

Listing 53: Typical console output when executing two Python unit tests. Parsing the output is error prone and unneeded, as the results can be accessed directly.

```python
# dependency_eval/loop.py
def run_loop(
    model_configurations: List[ModelConfiguration],
    dataset: Dataset,
    model_configuration_finished_cb: Callable[[ModelConfiguration], None],
    item_cb: Callable[[ModelConfiguration, Dict[str, Any]], None],
):
    for model_configuration in tqdm(model_configurations):
        tqdm.write(f"Running for {model_configuration.name}")
        for item in tqdm(dataset.items):
            item_cb(model_configuration, item)
        model_configuration_finished_cb(model_configuration)
```

Listing 54: The general evaluation loop, which combines several each model configuration with a dataset task.

parameters can be introspected, enabling complex test scenarios. Each test scenarios tests certain properties. For the scenario in Listing 52 the tests assert that the parameter tab_behavior is used, while tab_behaviour is incorrect, as it is deprecated.

Evaluation Loop

At the heart of the evaluation pipeline lies the evaluation loop, which is located in dependency_eval/loop.py. It is abstractly defined in Listing 54:

The loop combines every model configuration with every dataset task. The progress is displayed using the tqdm dependency, which provides an estimate of the remaining run time, as seen in Listing 55:

The passed item callback is called for every different combination. By abstracting the content of the item callback, several scenarios are possible, such as only running the neural code completion, evaluation, or both. After the whole dataset for a single model configuration was processed, the model configuration finished callback is called, running any finishing task, such as saving the results for the configuration.

Robust Neural Code Completion

The completion is located in dependency_eval/generate.py. It requires a project directory with a virtual environment, in which all dependencies are installed.

The code for the passed task is built, excluding the gold solution and then written to the file code.py in the project directory. In addition, a LSPGenerationConfig object is created and serialized into JSON. Depending on if the code should be completed using the approach or not, the field enabled is toggled. Then a subprocess runs the Python binary of the project virtual environment, calling the file dependency_eval/eval_prompt.py and passing the LSPGenerationConfig, the name of the model and the path to the code.py file as arguments. This trick enables running the neural code completion inside the exact

```
❯ python dependency_eval all
 67%|                                        | 2/ 3 [1:44:02<34:24, 2064.64s/it]
100%|                                        | 22/22 [ 42:46<00:00,  116.65s/it]
```

Listing 55: Sample output of the running evaluation. The first progress bar displays the progress in evaluation whole model configurations. The lower progress bar shows the progress of a single model configuration, showing the progress in evaluation a single task.

environment as defined in the dataset. This ensures that only these dependencies are found and used, without contamination by the evaluation pipeline dependencies.[1]

Similar to before, eval_prompt.py suppressed the standard output, as it prints the completed code into the standard output.

Isolating the generation from the evaluation pipeline process increases the robustness, which is vital as the pipeline may run for several hours. Ff the generation should fail due to out of memory issues, issues in the communication with the language server or other places, the evaluation pipeline can resume.

Code Evaluation Sandbox

To securely evaluate foreign code, a special evaluation sandbox is created using Docker. The implementation currently only uses plain Docker. As Docker is a containerization tool, it is a vast improvement over the reliability guard of e.g. HumanEval+. But improvements can still be made by adding a tool such as https://github.com/google/gvisor°, to further secure the kernel.

IPC with the evaluation sandbox again relies on sending JSON serialized messages over the standard output stream. Similar to running the code completion, the evaluation starts a subprocess, passes all arguments and handles errors. Finally, all dangling resources are removed.

The Docker container is started with the following arguments seen in Listing 56.

The Docker container image is an official Python image, where the Python version is set from the task. Ultimatively this creates the specific environment needed for the task. The Python package dependencies are installed inside of the container, before running the

```python
# dependency_eval/eval.py
def get_docker_cmd(item: Dict[str, Any], code_file: str, requirements_file: str):
    return [
        "docker",
        "run",
        "-it",
        "--rm",
        "--name",
        "dev_dataset_eval_item_" + SALT,
        "-v",
        f"{requirements_file}:/tool/requirements.txt",
        "-v",
        f"{code_file}:/code/llm_lsp_code.py",
        "-w",
        "/code",
        "--cpus",
        "1",
        # "--network", "none", # TODO: use docker build
        get_docker_image(item["python_version"]),
        "sh",
        "-c",
        "python -m venv /tool/venv && /tool/venv/bin/pip install -r /tool/requirements.txt 2>error.log >/dev/null && /tool/venv/bin/python llm_lsp_code.py || cat error.log",
    ]
```

Listing 56: Arguments to Docker run, which starts the evaluation in a Docker container.

[1]The language server might be able to find the virtual environment, based on the passed code file alone. But the custom logic to determine if a symbol is deprecated uses importlib, reusing the dependencies available in the currently running Python instance.

```python
# dependency_eval/constants.py
SALT = str(uuid4())[:8]
```

Listing 57: The salt is a unique identifier appended to any name used during the evaluation.

evaluation. The code to evaluate is mounted into the container, next to the requirements file. The compute is limited to a single CPU.

Docker supports a filesystem size limit using the **--storage-opt** flag, which could be used to limit the size of the filesystem. In practice, this requires support by the Docker storage driver, of which filesystems like **btrfs** and **zfs** are favoured[55]. This implementation does not limit the filesystem size for compatibility reasons.

The container execution time is limited to two minutes, which is enough time to install larger dependencies, such as PyTorch. After the two minutes have elapsed, the container is forcefully removed if it still runs. To reference the container, it is given the name **dev_dataset_eval_item_**, followed by a salt. The salt is unique to the run and obtained by taking the first 8 characters of a random UUID4:

This allows for multiple evaluations to run in parallel, without interfering with each other.

Test Metric Calculation

TODO: Move to evaluation?

Virtual Environment Management

Running the neural code completion requires a whole Python virtual environment per task, as each task has its own pinned dependencies. As creating each environment may take several minutes, especially when installing larger dependencies, the environments are cached and reused.

The dependencies are first sorted, then hashed using SHA256, to further enable reuse between tasks. The hash is used as a name for the virtual environment. If the environment

```
❯ ncdu .venv-cache
ncdu 1.15.1 ~ Use the arrow keys to navigate, press ? for help
--- .venv-cache ----------------------------------------------------
  5.5 GiB [##########] /2f6047e131995d68dafa2008945d7273e...
  5.4 GiB [######### ] /1bd45cb310f62185156a456a81d82330b...
  5.4 GiB [######### ] /e364c14c308dd832c2b81578a97832956...
  5.3 GiB [######### ] /ffbe7261e69b4907732e7ae3703238a2f...
  5.3 GiB [######### ] /6d9e208d9fcaa2c0e5d01c5445940ca59...
  5.3 GiB [######### ] /b1eaec1a8fc60ab556d0956740c5efb2f...
  5.3 GiB [######### ] /fafdab0ca240001968800f4cec74e74e7...
  5.3 GiB [######### ] /def6538d1fbe60c84db770d2a0eb1409d...
  5.3 GiB [######### ] /8e366a4d6499d4161b4de858297c95800...
  5.3 GiB [######### ] /4b54a3e088f036ffce13fc33266e54896...
  5.3 GiB [######### ] /431dcf3698ac8ebf3daa17806ca307690...
  5.3 GiB [######### ] /18985a70f9fca803b0611deac5a5da771...
  5.3 GiB [######### ] /5e80e6718a50c85b6f542793145cdde4c...
  5.3 GiB [######### ] /d537a621b87055e720f03f58d51e76f9a...
  5.3 GiB [######### ] /596f23bf62197f5329237fa8d69c9cc32...
  5.2 GiB [######### ] /4f36555d40130595a483d45ed8fa3f422...

Total disk usage:  84.8 GiB  Apparent size:  83.6 GiB  Items: 597956
```

Listing 58: Space used by the virtual environments in the cache. Results obtained using **ncdu**. Each directory is a Python virtual environment containing specific dependencies and the installed approach.

```python
# dependency_eval/venv_cache.py
from venv import EnvBuilder
def create_venv(venv_directory: str, requirements: str):
    tqdm.write("Creating new venv in cache")
    builder = EnvBuilder(
        system_site_packages=False,
        clear=False,
        symlinks=False,  # Important for the Docker container!
        upgrade=False,
        with_pip=True,
        prompt=None,
        upgrade_deps=False,
    )
    builder.create(venv_directory)
    context = builder.ensure_directories(venv_directory)
    with open(REQUIREMENTS_FILE, "w") as f:
        f.write(requirements)
    cmd = [context.env_exec_cmd, "-m", "pip", "install", "packaging", "wheel"]
    subprocess.check_output(cmd, stderr=subprocess.STDOUT)
    cmd = [context.env_exec_cmd, "-m", "pip", "install", "-r", REQUIREMENTS_FILE]
    subprocess.check_output(cmd, stderr=subprocess.STDOUT)
    os.remove(REQUIREMENTS_FILE)
```

Listing 59: Creating a virtual environment programmatically in Python.

does not exist yet, it is created using the venv package. Instead of the typical CLI use, the following Python functions are used, as seen in Listing 59:

The EnvBuilder in Listing 59 creates the base environment without further dependencies. After manually installing packaging and wheel, the remaining dependencies can be installed using a regular pip install, using the Python installation of the virtual environment.

If the virtual environment exists yet, it is cloned into the code directory, such that the neural code completion can find the environment. As virtual environments contain links, copying is non trivial and involves more than copying the directory. Thus, the dependency clonevirtualenv is used to clone the cached environment.

The environments may be prepopulated once before evaluation, by running the subcommand create_venvs.

Main

The main file has the following structure, as seen in Listing 60:

The CLI is defined through the click dependency and invoked by calling the cli function. By using the function decorator click.group, the general CLI is defined, with the given version VERSION. It automatically detects the package name and sets it as the program name.

Subcommands are registered, by using the cli.command decorator on single functions, where each function then represents a single subcommand. Single arguments are defined through the click.option decorator.

The create_venvs function defined in Listing 60 defines a subcommand with the same name, taking three arguments, of which two have a default value. All uppercase variables are constants either defined in dependnecy_eval/constants.py, or in other parts of the file.

```python
# dependency_eval/__main.py__
import click

@click.group()
@click.version_option(VERSION)
@click.pass_context
def cli(ctx):
    ctx.obj = None


@cli.command()
@click.option("--dataset-file", default=DEFAULT_DATASET_PATH)
@click.option("--llm-lsp-directory", required=True)
@click.option("--venv-cache-directory", default=DEFAULT_VENV_CACHE_DIRECTORY)
@click.pass_obj
def create_venvs(
    args, dataset_file: str, llm_lsp_directory: str, venv_cache_directory: str
):
    dataset = load_dataset(dataset_file)
    for item in tqdm(dataset.items):
        tqdm.write(item["task_name"])
        get_venv_for_item(venv_cache_directory, None, llm_lsp_directory, item)


if __name__ == "__main__":
    cli()
```

Listing 60: Structure of the DependencyEval main file, which includes the CLI commands.

5 Results

TODO: Simpler languages like LISP easier to integrate auto completion, as less syntactical sugar? (example where instead of object.method it uses {} in python)

5.1 Collection method

5.2 Possible configurations

5.3 Results

Table 9: Research questions guiding the thesis.

Identifier	Keyword	Description
RQ1	Information source selection	Which sources provide CLMs with the necessary information to suggest fitting code?
RQ2	Context processing	How to filter sourced information to provide the most relevant information to CLMs?
RQ3	Context integration	How to integrate the contextual information into the neural code completion?
RQ4	Robustness	How to ensure the robustness of the approach?
RQ5	Generalizability	How to generalize the approach to work with different programming languages?
RQ6	Performance	How to reduce the performance impact of the approach?

6 Evaluation

6.1 Research Questions

6.2 Evaluation difficulties

6.3 Evaluation

7 Conclusion

7.1 Recap

7.2 Outlook

Acronyms

ABC:	Abstract Base Class
BOS:	Begin of Sequence
BPE:	Byte Pair Encoding
CLI:	Command Line Interface
CLM:	Causal Language Model
Code LLM:	Large Language Model for Code
EOS:	End of Sequence
IBMS:	Interrupt-based meta-strategy
IDE:	Integrated Development Environment
IPC:	Inter Process Communication
JSON:	JavaScript Object Notation
JSON-RPC:	JavaScript Object Notation-Remote Procedure Call
LLM:	Large Language Models
LM:	Language Model
LRU:	Least Recently Used
LSP:	Language Server Protocol
MBPP:	Mostly Basic Python Problems
MGD:	Monitor-Guided Decoding
MIME:	Multipurpose Internet Mail Extensions
PAD:	Padding
PEP:	Python Enhancement Proposal
PyPI:	Python Package Index
RAG:	Retrieval Augmented Generation
RPC:	Remote Procedure Call
URI:	Uniform Resource Identifier

Bibliography

[1] S. Amann, S. Proksch, S. Nadi, and M. Mezini, "A Study of Visual Studio Usage in Practice," in *2016 IEEE 23rd International Conference on Software Analysis, Evolution, and Reengineering (SANER)*, Suita: IEEE, Mar. 2016, pp. 124–134. doi: 10.1109/SANER.2016.39°.

[2] A. Ziegler *et al.*, "Productivity Assessment of Neural Code Completion," in *Proceedings of the 6th ACM SIGPLAN International Symposium on Machine Programming*, San Diego CA USA: ACM, Jun. 2022, pp. 21–29. doi: 10.1145/3520312.3534864°.

[3] L. A. Agrawal, A. Kanade, N. Goyal, S. K. Lahiri, and S. K. Rajamani, "Monitor-Guided Decoding of Code LMs with Static Analysis of Repository Context."

[4] "Speech and Language Processing." Accessed: Jun. 04, 2024. [Online]. Available: https://web.stanford.edu/~jurafsky/slp3/°

[5] "ChatGPT." Accessed: Jun. 10, 2024. [Online]. Available: https://chatgpt.com/°

[6] "https://home.csulb.edu/~cwallis/382/readings/482/mccolloch.logical.calculus.ideas.1943.pdf." Accessed: Jun. 10, 2024. [Online]. Available: https://home.csulb.edu/~cwallis/382/readings/482/mccolloch.logical.calculus.ideas.1943.pdf°

[7] L. Weng, "Controllable Neural Text Generation." Accessed: Jun. 12, 2024. [Online]. Available: https://lilianweng.github.io/posts/2021-01-02-controllable-text-generation/°

[8] T. B. Brown *et al.*, "Language Models Are Few-Shot Learners." Accessed: Jun. 12, 2024. [Online]. Available: http://arxiv.org/abs/2005.14165°

[9] J. Wei *et al.*, "Chain-of-Thought Prompting Elicits Reasoning in Large Language Models." Accessed: Oct. 10, 2023. [Online]. Available: http://arxiv.org/abs/2201.11903°

[10] S. Yao *et al.*, "Tree of Thoughts: Deliberate Problem Solving with Large Language Models." Accessed: Oct. 10, 2023. [Online]. Available: http://arxiv.org/abs/2305.10601°

[11] S. Rabanser, O. Shchur, and S. Günnemann, "Introduction to Tensor Decompositions and Their Applications in Machine Learning." Accessed: Jun. 20, 2024. [Online]. Available: http://arxiv.org/abs/1711.10781°

[12] "Tensor Attributes — PyTorch 2.2 Documentation." Accessed: Jul. 01, 2024. [Online]. Available: https://pytorch.org/docs/2.2/tensor_attributes.html°

[13] "Torch.Tensor — PyTorch 2.2 Documentation." Accessed: Jul. 01, 2024. [Online]. Available: https://pytorch.org/docs/2.2/generated/torch.tensor.html#torch.tensor°

[14] "IEEE Standards Association." Accessed: Jun. 20, 2024. [Online]. Available: https://standards.ieee.org/°

[15] "BFloat16: The Secret to High Performance on Cloud TPUs." Accessed: Jun. 20, 2024. [Online]. Available: https://cloud.google.com/blog/products/ai-machine-learning/bfloat16-the-secret-to-high-performance-on-cloud-tpus°

[16] "Torch.Tensor.to — PyTorch 2.3 Documentation." Accessed: Jul. 01, 2024. [Online]. Available: https://pytorch.org/docs/stable/generated/torch.Tensor.to.html°

[17] "Torch.Tensor.Dim — PyTorch 2.2 Documentation." Accessed: Jul. 01, 2024. [Online]. Available: https://pytorch.org/docs/2.2/generated/torch.Tensor.dim.html°

[18] “Torch.Tensor.Shape — PyTorch 2.2 Documentation.” Accessed: Jul. 01, 2024. [Online]. Available: https://pytorch.org/docs/2.2/generated/torch.Tensor.shape.html

[19] “Tensor Indexing API — PyTorch Main Documentation.” Accessed: Jul. 01, 2024. [Online]. Available: https://pytorch.org/cppdocs/notes/tensor_indexing.html

[20] “ Transformers.” Accessed: Jun. 18, 2024. [Online]. Available: https://huggingface.co/docs/transformers/v4.41.3/en/index

[21] “Models.” Accessed: Jul. 01, 2024. [Online]. Available: https://huggingface.co/docs/transformers/v4.41.3/en/main_classes/model

[22] “Auto Classes.” Accessed: Jul. 01, 2024. [Online]. Available: https://huggingface.co/docs/transformers/v4.41.3/en/model_doc/auto

[23] “Tokenizer.” Accessed: Jul. 01, 2024. [Online]. Available: https://huggingface.co/docs/transformers/v4.41.3/en/main_classes/tokenizer

[24] “Configuration.” Accessed: Jul. 01, 2024. [Online]. Available: https://huggingface.co/docs/transformers/v4.41.3/en/main_classes/configuration

[25] “Generation.” Accessed: Jul. 01, 2024. [Online]. Available: https://huggingface.co/docs/transformers/v4.41.3/en/main_classes/text_generation

[26] “Utilities for Generation.” Accessed: Jul. 01, 2024. [Online]. Available: https://huggingface.co/docs/transformers/v4.41.3/en/internal/generation_utils

[27] “GNU Emacs - GNU Project.” Accessed: Jul. 01, 2024. [Online]. Available: https://www.gnu.org/software/emacs/

[28] “LSP Mode - Language Server Protocol Support for Emacs - LSP Mode - LSP Support for Emacs.” Accessed: Jul. 01, 2024. [Online]. Available: https://emacs-lsp.github.io/lsp-mode/

[29] “Visual Studio Code - Code Editing. Redefined.” Accessed: Jul. 01, 2024. [Online]. Available: https://code.visualstudio.com/

[30] “Language Server Extension Guide.” Accessed: Jul. 01, 2024. [Online]. Available: https://code.visualstudio.com/api/language-extensions/language-server-extension-guide

[31] “Home - Neovim.” Accessed: Jul. 01, 2024. [Online]. Available: https://neovim.io/

[32] “Lsp - Neovim Docs.” Accessed: Jul. 01, 2024. [Online]. Available: https://neovim.io/doc/user/lsp.html

[33] “Zed - Code at the Speed of Thought.” Accessed: Jul. 01, 2024. [Online]. Available: https://zed.dev/

[34] “Adding New Languages - Zed.” Accessed: Jul. 01, 2024. [Online]. Available: https://zed.dev/docs/adding-new-languages

[35] “JetBrains IDEs: Enjoy an Exceptional Developer Experience.” Accessed: Jul. 01, 2024. [Online]. Available: https://www.jetbrains.com/ides/

[36] “Language Server Protocol (LSP) | IntelliJ Platform Plugin SDK.” Accessed: Jul. 01, 2024. [Online]. Available: https://plugins.jetbrains.com/docs/intellij/language-server-protocol.html

[37] “Helix-Editor/Helix.” Accessed: Jul. 01, 2024. [Online]. Available: https://github.com/helix-editor/helix

[38] "Language Support." Accessed: Jul. 01, 2024. [Online]. Available: https://docs.helix-editor.com/lang-support.html

[39] "Specification." Accessed: Jul. 01, 2024. [Online]. Available: https://microsoft.github.io/language-server-protocol/specifications/lsp/3.17/specification/

[40] T. Berners-Lee, R. T. Fielding, and L. M. Masinter, "Uniform Resource Identifier (URI): Generic Syntax," Jan. 2005. doi: 10.17487/RFC3986.

[41] F. Zhang *et al.*, "RepoCoder: Repository-Level Code Completion Through Iterative Retrieval and Generation." Accessed: Dec. 05, 2023. [Online]. Available: http://arxiv.org/abs/2303.12570

[42] S. Zhou, U. Alon, F. F. Xu, Z. Wang, Z. Jiang, and G. Neubig, "DocPrompting: Generating Code by Retrieving the Docs." Accessed: Jun. 23, 2024. [Online]. Available: http://arxiv.org/abs/2207.05987

[43] "DevDocs." Accessed: Jul. 04, 2024. [Online]. Available: https://devdocs.io/

[44] "Stack Overflow Developer Survey 2023." Accessed: Jul. 03, 2024. [Online]. Available: https://survey.stackoverflow.co/2023/?utm_source=social-share&utm_medium=social&utm_campaign=dev-survey-2023

[45] J. Austin *et al.*, "Program Synthesis with Large Language Models." Accessed: Jul. 08, 2024. [Online]. Available: http://arxiv.org/abs/2108.07732

[46] M. Chen *et al.*, "Evaluating Large Language Models Trained on Code." Accessed: Oct. 18, 2023. [Online]. Available: http://arxiv.org/abs/2107.03374

[47] J. Liu, C. S. Xia, Y. Wang, and L. Zhang, "Is Your Code Generated by ChatGPT Really Correct?."

[48] X. Du *et al.*, "ClassEval: A Manually-Crafted Benchmark for Evaluating LLMs on Class-level Code Generation." Accessed: Jul. 07, 2024. [Online]. Available: http://arxiv.org/abs/2308.01861

[49] "Styleguide." Accessed: Jul. 08, 2024. [Online]. Available: https://google.github.io/styleguide/pyguide.html

[50] "PEP 287 – reStructuredText Docstring Format | Peps.Python.Org." Accessed: Jul. 08, 2024. [Online]. Available: https://peps.python.org/pep-0287/

[51] "Unittest — Unit Testing Framework." Accessed: Jul. 08, 2024. [Online]. Available: https://docs.python.org/3/library/unittest.html

[52] "PyPI · The Python Package Index." Accessed: Jul. 08, 2024. [Online]. Available: https://pypi.org/

[53] "Serialization - Pydantic." Accessed: Jul. 08, 2024. [Online]. Available: https://docs.pydantic.dev/latest/concepts/serialization/

[54] "Textual - Home." Accessed: Jul. 08, 2024. [Online]. Available: https://textual.textualize.io/

[55] "Docker Run | Docker Docs." Accessed: Jul. 08, 2024. [Online]. Available: https://docs.docker.com/reference/cli/docker/container/run/

[56] "Contextlib — Utilities for with-Statement Contexts." Accessed: Aug. 05, 2024. [Online]. Available: https://docs.python.org/3/library/contextlib.html

[57] "PEP 702 – Marking Deprecations Using the Type System | Peps.Python.Org." Accessed: Jul. 24, 2024. [Online]. Available: https://peps.python.org/pep-0702/°

[58] E. Imgrund, "Locating and Editing Knowledge in Large Transformer Models for Code Generation." Jul. 2024.

Listings

List of Figures

Figure 1: IEEE 754 half-precision floating point number: float16 11
Figure 2: Brain Floating Point Format: bfloat16 11
Figure 3: Generation strategies 15
Figure 4: Components of the IBMS 30
Figure 5: Format of a typical model generation result 31
Figure 6: Components of the LSP aided generation 34
Figure 7: The structure of the comment created by the “completion” interrupt type. The insert texts of all items are joined with “,”, similar to the colored bars. 57
Figure 8: The structure of the comment created by the “deprecation” interrupt type. For each item, a line with a hint is created, where the deprecation message is inserted at the position of the colored bar. 57
Figure 9: The structure of the comment created by the “signature” interrupt type. The green bar is replaced by the signature text of the item, while the pink text is replaced by the shortened documentation string. 57

List of Tables

Table 1: Relevant PyTorch data types 11
Table 2: Special tokens and token id attributes 13
Table 3: Generation configuration parameters 15
Table 4: Error codes and messages 19
Table 5: General requirements of the thesis 26
Table 6: Special requirements of the IBMS 29
Table 7: Special requirements of the dataset 37
Table 8: Dependencies in DependencyEval 40
Table 9: Research questions guiding the thesis. 70
Table 10: Possible language ids defined in the LSP specification 80

List of Code

Listing 1: Loading a pre-trained model 12
Listing 2: Using AutoClasses to load a model 13
Listing 3: Loading a tokenizer 13
Listing 4: Tokenizing text 13
Listing 5: Detokenizing a single sequence 14
Listing 6: Generating a sequence of tokens 14
Listing 7: Stopping criteria signature 16
Listing 8: Logits processor signature 16
Listing 9: General message interface 18
Listing 10: Request object format 18
Listing 11: Response object format 18
Listing 12: Notification object format 19
Listing 13: Position structure 19
Listing 14: TextDocumentItem structure 19
Listing 15: Initialize message parameters 20
Listing 16: DidOpenTextDocument message parameters 21
Listing 17: CompletionItem structure 21

Listing 18: CompletionItemLabelDetails structure 22
Listing 19: CompletionItemTag enum 22
Listing 20: MarkupContent structure 22
Listing 21: MarkupKind enum 22
Listing 22: Completion message parameters 22
Listing 23: CompletionContext structure 23
Listing 24: CompletionTriggerKind enum 23
Listing 25: CompletionList structure 23
Listing 26: SignatureHelp message parameters 23
Listing 27: SignatureHelp structure 24
Listing 28: SignatureInformation structure 24
Listing 29: ParameterInformation structure 24
Listing 30: Combination of fields to form the full solution in DependencyEval 39
Listing 31: Example code snippet for Pydantic showing the correct completion 40
Listing 32: Example code snippet for Textual showing the correct completion 41
Listing 33: Directory structure of the llm_lsp package. 43
Listing 34: The LspGenerationConfig data class, enabling customizable behavior. 44
Listing 35: The rearrange_according_to_beams method of the BeamTracker class, which reorders the items according to the sequence selections. 45
Listing 36: The BeamIndexStoringSearchScores class, which tracks the indices of selected sequences at each step and stores the final sequences before the final selection of the singular best sequence. 46
Listing 37: The InterruptStoppingCriteria class, which determines that the generation loop should be stopped when the interrupt token id occurs. 46
Listing 38: Entering the PyTorch device context manager before passing all arguments to the wrapped internal completion method. 47
Listing 39: The interrupt loop, which is the core of the Generator class and which enables the IBMS. Long argument lists are abbreviated by "..." 48
Listing 40: The format of the prompt used to predict the next symbol from the list of non deprecated completions directly. 50
Listing 41: The general structure employed for logging by the LogMixin. 51
Listing 42: The __call__ method of the LSPProcessor class. 51
Listing 43: Start of the scores_for_batch method, listing how the context retrieved from a language server. 53
Listing 44: The is_deprecated function, which determines if a completion item is deprecated. For speed, the results are memoized using a LRU cache. 54
Listing 45: The Interrupt dataclass, which is used to store the interrupt context and the interrupt token id. 55
Listing 46: The error message produced by PyTorch when the score of the interrupt token is set to the highest possible value. 56
Listing 47: A full sample prompt, including system and user prompt, comments and initial code. 57
Listing 48: The InterruptType abstract base class, which is used to implement the different interrupt types. 57
Listing 49: Waiting on to be returned completions from the language server. The asynchronous code is run in the synchronous code by retrieving the current outer event loop. 60
Listing 50: Directory structure of the dependency_eval repository. 61
Listing 51: The contents of the data/tasks/textual_1.py file, which is used to create the task textual_1 in the DependencyEval dataset. The file is split to obtain the imports, additional context, the function definition, function documentation string and gold standard completion. 61

Listing 52: The unit tests for the textual_1 task. The contents are stored in data/tests/textual_1.py. Next to the tested dependency, only the built-in unit test framework is used. The results are communicated via standard streams. 61
Listing 53: Typical console output when executing two Python unit tests. Parsing the output is error prone and unneeded, as the results can be accessed directly. 62
Listing 54: The general evaluation loop, which combines several each model configuration with a dataset task. 64
Listing 55: Sample output of the running evaluation. The first progress bar displays the progress in evaluation whole model configurations. The lower progress bar shows the progress of a single model configuration, showing the progress in evaluation a single task. 64
Listing 56: Arguments to Docker run, which starts the evaluation in a Docker container. 65
Listing 57: The salt is a unique identifier appended to any name used during the evaluation. 66
Listing 58: Space used by the virtual environments in the cache. Results obtained using ncdu. Each directory is a Python virtual environment containing specific dependencies and the installed approach. 66
Listing 59: Creating a virtual environment programmatically in Python. 67
Listing 60: Structure of the DependencyEval main file, which includes the CLI commands. 67
Listing 61: CompletionItemKind enum 81
Listing 62: Full JSON schema of the DependencyEval dataset. The schema defines the structure of each task entry in the distributed JSONL file. 81

List of Examples

Example 1: Language model predicting the last word of the famous English pangram. 4
Example 2: Natural language encoded into tokens. 5
Example 3: Enforcing "yes" or "no" answers 7
Example 4: Few-shot prompting 8
Example 5: Context injection 8

Appendix

Language Server Protocol

Language	Language ID
ABAP	abap
Windows Bat	bat
BibTeX	bibtex
Clojure	clojure
Coffeescript	coffeescript
C	c
C++	cpp
C#	csharp
CSS	css
Diff	diff
Dart	dart
Dockerfile	dockerfile
Elixir	elixir
Erland	erlang
F#	fsharp
Git	git-commit and git-rebase
Go	go
Groovy	groovy
Handlebars	handlebars
HTML	html
Ini	ini
Java	java
JavaScript	javascript
JavaScript React	javascriptreact
JSON	json
LaTeX	latex
Less	less
Lua	lua
Makefile	makefile
Markdown	markdown
Objective-C	objective-c
Objective-C++	objective-cpp
Perl	perl
Perl 6	perl6
PHP	php
Powershell	powershell
Pug	jade
Python	python
R	r
Razor	razor
Ruby	ruby
Rust	rust
SCSS	scss or sass
Scala	scala
ShaderLab	shaderlab
Shellscript	shellscript
SQL	sql
Swift	swift
TypeScript	typescript
TypeScript React	typescriptreact
TeX	tex
Visual Basic	vb
XML	xml
XSL	xsl
YAML	yaml

Table 10: Possible language ids defined in the LSP specification

```typescript
namespace CompletionItemKind {
    const Text: 1;
    const Method: 2;
    const Function: 3;
    const Constructor: 4;
    const Field: 5;
    const Variable: 6;
    const Class: 7;
    const Interface: 8;
    const Module: 9;
    const Property: 10;
    const Unit: 11;
    const Value: 12;
    const Enum: 13;
    const Keyword: 14;
    const Snippet: 15;
    const Color: 16;
    const File: 17;
    const Reference: 18;
    const Folder: 19;
    const EnumMember: 20;
    const Constant: 21;
    const Struct: 22;
    const Event: 23;
    const Operator: 24;
    const TypeParameter: 25;
}
type CompletionItemKind = 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 | 13 | 14 | 15 | 16 | 17 | 18 | 19 | 20 | 21 | 22 | 23 | 24 | 25;
```

Listing 61: CompletionItemKind enum

DependencyEval Dataset

```json
{
  "$schema": "https://json-schema.org/draft/2020-12/json-schema-core",
  "type": "object",
  "properties": {
    "task_id": {
      "type": "string",
      "pattern": "^PackageEval_\\d+$"
    },
    "task_name": {
      "type": "string",
      "pattern": "^.*_\\d+$"
    },
    "test_code": {
      "type": "string"
    },
    "import_statements": {
      "type": "array",
      "items": {
        "type": "string"
      }
    },
    "package_dependencies": {
      "type": "array",
```

```
      "items": {
        "type": "string"
      }
    },
    "context": {
      "type": "string"
    },
    "function_signature": {
      "type": "string"
    },
    "function_documentation": {
      "type": "string"
    },
    "entry_point": {
      "type": "string"
    },
    "solution": {
      "type": "string"
    },
    "reason": {
      "type": "string"
    },
    "kind": {
      "type": "string"
    },
    "date": {
      "type": "string",
      "format": "date"
    },
    "code_kind": {
      "type": "string"
    },
    "modification_kind": {
      "type": "string"
    },
    "changelog": {
      "type": "string",
      "format": "uri"
    },
    "python_version": {
      "type": "string"
    }
  },
  "required": [
    "task_id",
    "task_name",
    "test_code",
    "import_statements",
    "package_dependencies",
    "context",
    "function_signature",
    "function_documentation",
    "entry_point",
    "solution",
    "reason",
    "kind",
```

```
    "date",
    "code_kind",
    "modification_kind",
    "changelog",
    "python_version"
  ]
}
```

Listing 62: Full JSON schema of the DependencyEval dataset. The schema defines the structure of each task entry in the distributed JSONL file.

QR code



Scan the QR code to access the source code of this thesis:
https://github.com/data-niklas/master_thesis°

Cite this thesis:

BibTex

```bibtex
@thesis{niklas_loeser_2024,
  title = {Interrupt-Guided Neural Code Completion with Static Analysis of Dependencies},
  author = {Niklas Loeser},
  year = {2024},
  month = {9},
  school = {Heidelberg University},
  url = {https://github.com/data-niklas/master_thesis},
}
```